

From Legal Text to Legal Graphs: Judgment Prediction with Graph Neural Networks

Ziv Barretto

April 20, 2026

Abstract

The growing imbalance between the volume of court cases and the capacity of the judicial system has increased interest in automated methods that can assist legal analysis and outcome prediction. In the Indian setting, this problem is especially challenging because judgments are long, noisy, often multilingual, and frequently spread across multiple hearings. Prediction pipelines using text embeddings risk leaking the final decision through operative text and identity shortcuts. This paper presents a graph embedding based framework for Legal Judgment Prediction over Indian court decisions. The approach constructs a heterogeneous knowledge graph from raw judgments using legal named entities, rhetorical-roles of texts and outcome labels after multi-hearing case consolidation. The approach caters to multilingual documents. By representing cases alongside parties, courts, statutes, provisions, precedents, and argument structures, the model captures both the internal logic of a case and the shared legal relationships across the corpus. Additionally, we introduce a comprehensively annotated dataset of over 73,000 Indian court cases spanning five legal domains, which will be released to support downstream legal NLP research. The proposed framework, using a Heterogeneous Graph Transformer, achieves 80% accuracy and a 0.795 macro-F1 score in the pooled cross-domain setting. The results demonstrate that graph-based representations outperform text-only baselines, offering a robust, interpretable alternative that successfully resists identity-based leakage shortcuts.

Contents

1	Introduction	4
1.1	The Access to Justice Problem in India	4
1.2	What is Legal Judgement Prediction?	5
1.3	Why LJP is Hard	5
1.4	Why Graph-Based Approaches?	6
1.5	Contributions of This Work	6
1.6	Thesis Organisation	7
2	Related Work	8
2.1	Feature Engineering and Statistical Models	8
2.2	Text-Based Deep Learning	9
2.3	Graph-Based Approaches	9
2.4	Heterogeneous GNNs and the Motivation for HGT	10
2.5	Indian Legal NLP and OpenNyAI	11
2.6	Gaps This Work Addresses	11
3	System Overview and Data Pipeline	13
3.1	System Architecture Overview	13
3.2	Document Collection and Legal Domains	15
3.2.1	Source Selection and Crawling Strategy	15
3.2.2	Local Bucketing and Legal Domains	15
3.3	Named Entity Recognition and Rhetorical Role Classification .	16
3.3.1	The OpenNyAI Pipeline	16
3.3.2	Entity Types and Their Graph Roles	17
3.3.3	Rhetorical Role Taxonomy and Leakage-Safe Filtering .	18
3.4	Outcome Labelling with Mistral	18
3.4.1	Labelling Methodology	18
3.4.2	Coverage and Label Distribution	19

3.5	Multi-Hearing Merging	20
3.5.1	The Problem	20
3.5.2	Merge Strategy	20
3.6	Technology Stack	21
4	Heterogeneous Knowledge Graph Construction	23
4.1	Motivation: Why a Heterogeneous Graph?	23
4.2	Graph Schema	24
4.2.1	Node Types	24
4.2.2	Cross-Case Sharing	25
4.2.3	Edge Types	25
4.2.4	Attributes	26
4.2.5	Visualizing the Graph Structure	27
4.3	Leakage Prevention Strategy	27
4.4	Text Embedding and Feature Extraction	31
4.4.1	The bge-m3 Text Encoder	31
4.4.2	Node Feature Assembly	31
5	Graph Exploration and Structural Insights	33
5.1	Why Explore the Graph Before Prediction?	33
5.2	Graph Statistics and Structural Analysis	34
5.2.1	Overall Graph Statistics	34
5.2.2	Node Type Distribution	34
5.2.3	Degree Distribution and Hub Entities	36
5.2.4	Cross-Domain Bridge Analysis	37
5.2.5	Entity Sharing Heatmap Across Domains	39
5.2.6	Connectivity Score Distribution	40
5.3	Per-Domain Corpus Analysis	40
5.3.1	Case Length and Sentence Distributions	41
5.3.2	Entity Richness Per Domain	41
5.3.3	Rhetorical Role Composition Per Domain	43
5.3.4	Outcome Distribution Per Domain	43
5.4	Key Structural Insights	44
6	Prediction Model and Results	45
6.1	Why a Heterogeneous Graph Transformer?	45
6.2	Model Architecture	45
6.3	Experimental Setup	46

6.4	Ablation Variants and Rationale	47
6.5	Baseline Comparisons	48
6.6	Cross-Bucket Results	48
6.7	Per-Domain Results	50
6.8	Ablation Findings	51
6.9	Probing Classifier Analysis	52
6.10	Attention and Saliency Analysis	53
6.11	SHAP Feature Importance	55
6.12	Embedding Visualisation (t-SNE)	55
6.13	Error Analysis	58
7	Discussion	60
7.1	Summary of Findings	60
7.2	Limitations	62
7.3	Future Work	63
8	Conclusion	66

Chapter 1

Introduction

1.1 The Access to Justice Problem in India

Access to justice in India is not only a legal ideal but also a large-scale administrative challenge. Court backlogs remain extremely high across the judicial system, which means that delay itself becomes a serious barrier to justice. When cases take years to move, litigation becomes more expensive, evidence becomes harder to trace, and ordinary citizens are less able to sustain meaningful participation in the legal process.

This does not mean that courts should be automated. Judicial decision-making involves interpretation, justification, and public accountability in ways that no prediction system can replace. A more realistic goal is decision support: computational tools that help organise large volumes of case material, surface patterns across past disputes, and assist legal research or case preparation. Legal Judgment Prediction (LJP) sits within that narrower and more defensible ambition.

The Indian setting makes this especially important. Judgments are often long, noisy, citation-heavy, and spread across multiple hearings. As a result, the challenge is not just to classify a document, but to build a faithful representation of a dispute from complex judicial text. This thesis addresses that challenge through structured case modelling rather than flat document classification.

1.2 What is Legal Judgement Prediction?

Legal Judgment Prediction is the task of estimating some aspect of a court outcome from information contained in the case record [1, 8, 12]. Different papers define the target differently: some predict violation versus non-violation, some predict appeal outcomes, and others predict charges or penalties. In all cases, the core idea is the same: given a representation of a dispute, can a model forecast the judicial result?

In this thesis, LJP is defined narrowly as outcome prediction over Indian court judgments after removing explicit decision-bearing text. The aim is not to generate legal reasoning or normative conclusions, but to test whether pre-decision signals — facts, arguments, statutes, provisions, and precedents — can support outcome prediction when represented in a structured way.

This definition matters because much of the LJP literature has been criticised for hidden leakage [13]. If a model can read operative text, ruling sections, or highly outcome-correlated metadata, then the task becomes answer recovery rather than prediction. For that reason, LJP is as much an experimental-design problem as it is a modelling problem. The central question is not only *what* to predict, but also *what information the model is allowed to use*.

1.3 Why LJP is Hard

LJP is difficult because legal judgments are not ordinary classification inputs. They are long, internally heterogeneous documents in which facts, procedural history, lawyer submissions, precedent citations, and judicial reasoning are interwoven across many pages. Standard text models often truncate or simplify this structure, which risks losing legally decisive context [3, 11].

The problem is harder in India because the text is noisy and structurally inconsistent. Judgments contain abbreviations, citation variations, OCR errors, transliterated names, and multilingual or mixed-register language. Core legal entities such as statutes, provisions, courts, and parties rarely appear in one clean canonical form. On top of that, one dispute may generate several hearing-stage documents rather than a single final text. If these are treated as separate cases, both the input representation and the labels become unreliable.

A further difficulty is that legal reasoning is inherently relational. Out-

comes depend not just on isolated sentences, but on how facts, arguments, statutes, provisions, precedents, and parties connect. At the same time, LJP is highly vulnerable to leakage. Judgments often contain phrases such as *appeal allowed* or *petition dismissed*, and even identity-heavy metadata may act as shortcuts. Finally, the task is cross-domain: different legal areas have different statutes, document styles, and outcome distributions. Together, these challenges make LJP a representation and evaluation problem first, and only then a modelling problem.

1.4 Why Graph-Based Approaches?

A graph-based approach is a natural fit for legal disputes because cases are structured collections of typed objects and relations. A case is heard in a court, involves parties and lawyers, cites statutes and precedents, and contains argument blocks linked to different sides. When all of this is flattened into one sequence, the model must recover structure that is already present in the source material. A graph preserves that structure explicitly.

Graphs are useful here for two reasons. First, they preserve heterogeneity: a statute, a party, and a facts section do not play the same legal role and should not be treated as identical inputs. Second, they allow cross-case reasoning through shared legal objects. If two cases cite the same statute or precedent, they are connected in a legally meaningful way, not merely through surface lexical similarity. This thesis therefore argues for a heterogeneous, leakage-audited, cross-case graph representation for Indian LJP.

1.5 Contributions of This Work

This thesis makes five main contributions.

1. It constructs a large structured corpus of Indian court cases spanning five legal domains, with more than 73,000 final-case records after multi-hearing consolidation.
2. It develops an end-to-end pipeline that converts raw judgments into leakage-audited, graph-ready representations using document collection, OpenNyAI enrichment, outcome labelling, multi-hearing merging, and explicit leakage filtering.

3. It proposes a reasoning-focused heterogeneous knowledge graph schema in which cases are represented together with text blocks, parties, courts, statutes, provisions, and precedents, while legally meaningful sharing is separated from identity-based shortcuts.
4. It applies a Heterogeneous Graph Transformer to this graph and evaluates it in the pooled cross-domain setting, showing that structured graph representations outperform text-only baselines on the thesis dataset.
5. It complements prediction with graph analysis, ablations, and leakage-oriented reasoning, treating the graph not just as model input but as an empirical object that helps explain both performance and limitations.

1.6 Thesis Organisation

The remainder of the thesis is organised as follows. Chapter 2 reviews prior work on legal judgment prediction, transformer-based legal NLP, graph methods, heterogeneous GNNs, and Indian legal NLP resources. Chapter 3 describes the full data pipeline, from document collection to OpenNyAI enrichment, outcome labelling, and multi-hearing merging. Chapter 4 presents the heterogeneous graph schema and the leakage-prevention strategy. Chapter 5 studies the graph structurally through statistics, bridge analysis, and per-domain patterns. Chapter 6 introduces the prediction model and reports the main results and ablations. Chapter 7 discusses implications, limitations, and future directions. Chapter 8 concludes the thesis.

Chapter 2

Related Work

2.1 Feature Engineering and Statistical Models

The earliest wave of legal outcome prediction relied on feature engineering and classical statistical learning, representing cases using bag-of-words, TF-IDF vectors, or manually designed legal features, and training linear models, SVMs, or tree-based classifiers. Aletras et al. showed that n-gram and topic features extracted from ECHR case text could predict violation judgments with around 79% accuracy, with case facts emerging as the strongest predictive signal [1]. In the US context, Katz et al. built a time-evolving random forest over institutional metadata and historical voting patterns, achieving over 70% accuracy predicting Supreme Court outcomes across nearly two centuries [8]. Medvedeva et al. extended ECHR prediction work and demonstrated that predicting genuinely future cases from past ones substantially lowers performance, raising early concerns about temporal leakage [12].

These approaches remain relevant as strong, truncation-free baselines: TF-IDF classifiers can be competitive on long documents precisely because they avoid the input-length constraints that trouble neural models [11]. Their central limitation, however, is that flat lexical representations cannot capture discourse order, typed legal relations, or cross-case structure. A broader methodological critique by Medvedeva et al. found that the majority of LJP publications do not perform genuine prospective prediction, instead relying on unrealistic inputs or temporally inconsistent splits [13]. This critique is directly relevant here: the present thesis treats leakage control as a core methodological contribution.

2.2 Text-Based Deep Learning

The second major wave replaced manual features with learned neural representations. Domain-adapted transformers quickly became dominant; LEGAL-BERT demonstrated that pre-training on legal corpora—legislation, court cases, and contracts—yields consistent gains over generic BERT on legal classification tasks [3]. The LexGLUE benchmark consolidated several legal NLU datasets and confirmed that legal-domain models systematically outperform generic ones across diverse tasks [4].

The central weakness of standard transformers in law is document length. Most judgments far exceed the 512-token window of vanilla BERT, forcing researchers to truncate, use sliding windows, or adopt hierarchical architectures. Work on modified Longformer architectures shows that extending context to 8,192 sub-words improves performance on long legal classification tasks, but at substantial computational cost, and even then hierarchical TF-IDF variants remain competitive in some settings [11]. Hierarchical encoders are attractive because they mirror legal intuition—facts, arguments, and operative text play distinct roles—but they remain document-centric: they model internal case structure while ignoring cross-case links through shared statutes or precedents. The present thesis departs from purely text-based work by treating text embeddings as node features within a larger relational representation, rather than as the complete model.

2.3 Graph-Based Approaches

Graph-based methods were developed precisely to model the relational structure that flat text pipelines miss. TopJudge formalised dependencies among legal subtasks—law articles, charges, and penalties—as a directed acyclic graph, showing that joint prediction over structured outputs improves performance compared to treating each subtask independently [22]. Subsequent work expanded this relational view by incorporating retrieved precedents and law-article graphs, with more recent systems using LLM-and-retrieval pipelines to bring external legal scaffolding into prediction [21, 18].

Within Indian LJP, graph-based work is growing. Khatri et al. explored GNNs for Indian legal judgment prediction and raised important questions about the role of case-graph design and demographic shortcuts [9]. More recent large-scale Indian work has extended coverage and explored RAG-

style pipelines [14]. Existing graph approaches nonetheless leave gaps that this thesis addresses: many are built for civil-law charge-prediction datasets, rely on a single court, or use graph structure to improve prediction without comparable attention to decision-text or identity leakage. The present work is closest in spirit to this line but narrows its claim: the graph should improve prediction by exposing legal structure across cases while explicitly blocking shortcuts that rely on post-decision information.

2.4 Heterogeneous GNNs and the Motivation for HGT

When a legal problem is represented as a graph, the choice of GNN architecture matters because legal graphs contain multiple semantically distinct node and edge types. A homogeneous GNN is too blunt: a case node, a statute node, and a precedent node play different epistemic roles, and edges such as *cites_precedent* and *heard_in* should not be collapsed into a single message-passing rule.

R-GCN introduced relation-specific weight matrices for multi-relational graphs and is well-suited to knowledge graph completion tasks [19]. Its limitation for the present thesis is that relation-specific parameterisation can become coarse as the number of types grows, and it does not model attention over heterogeneous neighbourhoods. HAN addressed heterogeneity through hierarchical attention over meta-paths, learning node-level and semantic-level importance separately [20]. However, it requires manually specified meta-paths, which hard-codes assumptions that may not transfer across legal domains. HGT provides a more flexible alternative: it uses node-type-dependent projections and edge-type-dependent attention, allowing the model to learn how much weight to assign to different heterogeneous neighbours without requiring handcrafted meta-path templates [5]. For the legal graph constructed in this thesis—which includes many node types, many edge types, and a deliberately cross-domain structure—HGT fits the design requirements better than R-GCN or HAN.

2.5 Indian Legal NLP and OpenNyAI

Indian legal NLP developed later than English-language benchmarks for US or European law, in part due to data and tooling scarcity. Early work on rhetorical role identification showed that Indian judgments have a recoverable discourse structure—preamble, facts, arguments, ratio, and ruling—that is useful for downstream tasks but not explicitly marked in raw text [2]. This made legal document structuring a first-class problem rather than a preprocessing convenience.

OpenNyAI is the most practically important development for this thesis, bringing together legal named entity recognition, rhetorical-role prediction, and summarisation for Indian court judgments into an open, modular pipeline [7, 17, 16]. On the benchmarking side, ILDC introduced a large corpus of Indian Supreme Court cases for judgment prediction and explanation [10], while IL-TUR broadened the scope to a multi-task legal understanding and reasoning benchmark [6]. Newer rhetorical-role resources such as LegalSeg confirm that structural modelling continues to matter for Indian judgments [15].

These contributions are substantial, but they do not resolve the core challenge addressed here: how to represent Indian cases so that a model can exploit legal structure across cases without quietly learning identity or post-decision shortcuts. This thesis builds on the Indian legal NLP ecosystem rather than apart from it, combining OpenNyAI’s pipeline components with a leakage-audited heterogeneous graph.

2.6 Gaps This Work Addresses

The literature reviewed above shows that legal judgment prediction has advanced substantially, but along partly disconnected lines. Classical models demonstrated that outcomes are statistically predictable; transformer models improved semantic representation; graph methods showed the value of relational structure; Indian NLP resources made domain-specific preprocessing feasible. Yet most prior work sacrifices at least one of the following: *scale*, *cross-domain coverage*, *cross-case legal structure*, or *leakage-aware experimental design*.

What remains comparatively rare is an Indian LJP system that operates over a large multi-domain corpus, consolidates multiple hearings per case, extracts legal entities and rhetorical structure, builds an explicitly heteroge-

neous cross-case graph, and evaluates under a design intended to resist both decision-text leakage and identity-based shortcuts. That is the niche this thesis occupies. Its claim is narrow and, for that reason, defensible: when Indian court judgments are converted into a leakage-audited heterogeneous graph, cross-case legal structure becomes usable in ways that flat text pipelines cannot easily match.

Chapter 3

System Overview and Data Pipeline

This chapter describes the end-to-end pipeline that turns raw Indian court-judgment text into a labelled, leakage-audited, heterogeneous knowledge graph ready for Graph Neural Network (GNN) training. The pipeline was designed around three practical constraints: (i) the corpus is large and noisy, so every stage is a batch process with explicit caching and auditing; (ii) many of the signals a predictor would want (decisions, dispositions, operative paragraphs) are precisely the signals a legally honest experiment must remove, so leakage prevention has to be baked in rather than bolted on; and (iii) the downstream target is a graph, so text extraction and entity resolution have to produce outputs that can be reified as typed nodes and edges without lossy renormalisation later on.

3.1 System Architecture Overview

The system is organised as a six-stage pipeline. Each stage is an independently re-runnable batch process; intermediate artefacts are cached on disk so that an expensive step (OpenNyAI NER on tens of thousands of judgments, or bge-m3 embedding on millions of text spans) never has to be repeated when a later stage changes.

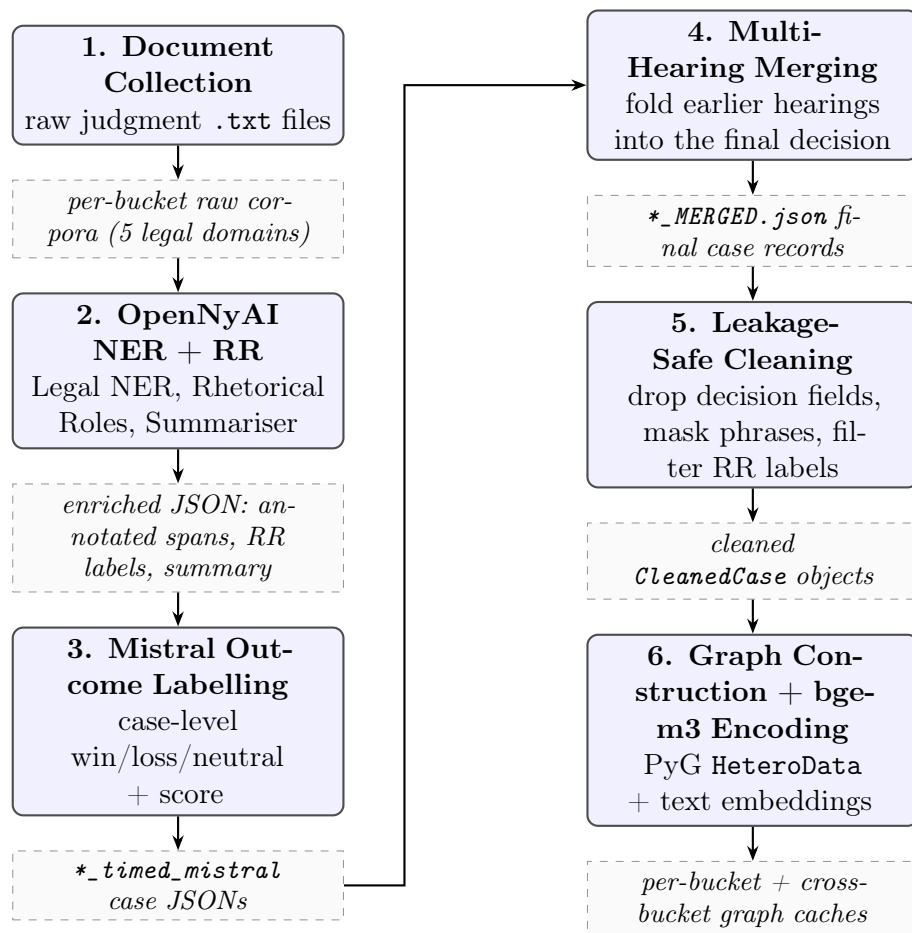


Figure 3.1: The six-stage data pipeline. Solid boxes are processing stages; dashed boxes are cached on-disk artefacts that feed the next stage.

Stages 1–3 are *textual enrichment*: they turn plain text into a structured JSON record with entities, rhetorical-role spans, and a case-level outcome label. Stage 4 reconciles the fact that a single legal matter often produces multiple hearing documents, so that each final case corresponds to one legal dispute rather than one PDF. Stage 5 is a leakage-safety gate: it explicitly removes the fields, sections, and phrases that would let a model trivially recover the outcome. Stage 6 reifies the cleaned records as a typed heterogeneous graph and attaches pre-computed **bge-m3** text embeddings to every text node. Chapter 4 details the schema produced by stage 6.

3.2 Document Collection and Legal Domains

3.2.1 Source Selection and Crawling Strategy

The data source for the underlying judgments is Indian Kanoon (indiankanoon.org). It was selected for its comprehensive, publicly accessible archive of Indian High Court judgments and its semi-structured search interface, which is uniquely suited for programmatic access compared to proprietary databases. However, accessing a deep, historical sample from the platform involves structural challenges. Indian Kanoon’s search results for any single query are practically truncated at a cap of 40 pages (approximately 400 accessible documents) per search window.

To overcome this truncation, data collection was structured around discrete *court-month windows*. For each target High Court and for each calendar month across the target years, the scraper executed an explicitly bounded query (e.g., `doctype:<court>fromdate:1-<month>-<year>todate:<last_day>-<month>-<year>`). By narrowing the temporal bounds to a single month per jurisdiction, the total number of judgments returned per window consistently stayed within the access limits, allowing for an exhaustive download of the raw corpus without relying on single broad keyword queries. The judgments were initially collected and stored directly in their raw PDF format. Furthermore, no deduplication methodologies were employed on the text or case level beyond a basic check to prevent re-downloading the same numeric Indian Kanoon document ID.

3.2.2 Local Bucketing and Legal Domains

Rather than trusting black-box search engines to selectively define the corpus—which could bias the dataset towards heavily-cited “landmark” cases while omitting routine baseline cases—the classification of documents into specific legal domains was implemented locally as a deterministic post-processing step.

After downloading the entire accessible window, each raw PDF underwent text extraction (via PyMuPDF) and was passed through a local filtering script. This script evaluated the text against a rigid set of regular expressions corresponding to required statutory provisions (e.g., “Section 498A IPC”, “Motor Vehicles Act”, “POCSO”) and contextual keyword constraints. Scraped texts that successfully matched a domain’s predefined rules were partitioned

into the respective local directory buckets.

The collected corpus resulting from this process is organised into five **legal domain buckets**, chosen so that each bucket is large enough to train a domain-specific graph and internally coherent in terms of statutory framework, party structure, and typical remedies. The buckets are:

- **Family & Matrimonial** (`family_matrimonial_timed_mistral`): matrimonial disputes, maintenance, custody, and dowry-related proceedings. Dominated by IPC 498A, Sections 482, 506, and the CrPC.
- **Financial Fraud** (`fin_fraud_timed_mistral`): cheating, forgery, dishonour, and economic offences. Dominated by IPC 420, 468, 471.
- **Land & Property** (`land_property_timed_mistral`): land acquisition, title, and property disputes. Dominated by the Land Acquisition Act 1894 and the Constitution of India.
- **Motor Accidents** (`motor_accidents_timed_mistral`): compensation claims and related appeals. Dominated by the Motor Vehicles Act 1988 and Section 166.
- **Sexual Offences** (`sexual_offences_timed_mistral`): offences against the person, including POCSO matters. Dominated by the IPC, CrPC, and POCSO Act.

Every downstream artefact – entity canonicalisation tables, rhetorical-role annotations, outcome labels, and graph caches – is produced both per bucket and in a unified *cross-bucket* view. This two-level organisation is what later makes cross-domain transfer and cross-bucket bridge analysis possible (Chapter 5).

3.3 Named Entity Recognition and Rhetorical Role Classification

3.3.1 The OpenNyAI Pipeline

Textual enrichment is performed with the OpenNyAI legal NLP stack, running locally on GPU. The production configuration uses:

- **Legal NER** with `ner_model_name="en_legal_ner_trf"`, sentence-level inference (`ner_do_sentence_level=True`) and the built-in post-processor (`ner_do_postprocess=True`);
- **Rhetorical_Role** classification, assigning each sentence a legal discourse label (PREAMBLE, FACTS, ARGUMENTS, RATIO, RPC, etc.);
- **Summarizer** executed after rhetorical-role classification, which produces the section-structured summary (PREAMBLE, FACTS, ARGUMENTS, and a decision block that is later discarded).

The environment is pinned (spaCy 3.2.4, pydantic 1.7.4, CUDA-enabled PyTorch and CuPy) and run GPU-only: silent CPU fallback is disabled so that a broken GPU configuration fails fast rather than silently degrading throughput. All model caches are redirected into project-local folders so that re-runs do not contaminate `$HOME`. Each input judgment yields a combined JSON with NER annotations, rhetorical-role spans, and the summary.

3.3.2 Entity Types and Their Graph Roles

OpenNyAI’s legal NER tags span several coarse types; the ones we reify as graph nodes are:

- **STATUTE** and **PROVISION** – statutes (e.g. IPC, CrPC, Motor Vehicles Act) and their sections (e.g. Section 482). Provisions are additionally linked to their parent statute via a `belongs_to_statute` edge.
- **PRECEDENT** – cited case references.
- **COURT** and **JUDGE** – the court the matter is heard in and the presiding bench.
- **PETITIONER**, **RESPONDENT**, **PETITIONER_LAWYER**, **DEFENCE_LAWYER**, **LAWYER** – typed party and counsel roles. Role assignment uses the section in which the mention first appears (preamble vs. argument blocks) combined with OpenNyAI’s entity labels.
- **ORG**, **GPE**, **DATE**, **CASE_NUMBER** – auxiliary types extracted by NER, retained in the cleaned JSON but excluded from the final reasoning-focused graph (Section 4.2).

Entities are canonicalised per bucket: surface forms are normalised (case, whitespace, punctuation), grouped, and re-projected as `canonical_name`. Canonicalisation is what allows the same statute or court to be a *single shared node* across the thousands of cases that mention it, which is the structural property that makes the graph useful for cross-case reasoning.

3.3.3 Rhetorical Role Taxonomy and Leakage-Safe Filtering

OpenNyAI’s rhetorical-role taxonomy is the hinge on which the leakage-safety story turns. Of the labels produced by the RR model, two are inherently decisive – **RPC** (Ratio and Precedent Cited, in practice the court’s operative paragraph) and **RLC** (Ruling by Lower Court). Keeping either of them would let a classifier read the outcome verbatim.

The cleaning stage therefore treats both labels as `FORBIDDEN_ANNOTATION_LABELS`: any annotation whose label set intersects `{RPC, RLC}` is dropped outright, along with any annotation whose summary section is `DECISION` or whose label string contains `"decision"`, `"disposition"`, `"operative"`, `"order"`, or `"rpc"`. Only the three non-decisive summary fields – `PREAMBLE`, `FACTS`, and `ARGUMENTS` – are retained as text sources for the graph’s text nodes. This is the first leakage gate; a second, phrase-level gate is described in Section 4.3.

3.4 Outcome Labelling with Mistral

3.4.1 Labelling Methodology

Case-level outcome labels are produced by prompting a Mistral LLM with the OpenNyAI summary and asking it to return a structured judgement: a label (win = +1, loss = -1, or `NEUTRAL`), a confidence-weighted `outcome_score` in $[-1, +1]$, and a short textual explanation. Labelling is done from the *summaries produced by OpenNyAI*, not from the raw judgment text, so that the label generator sees the same content the downstream model would see if no leakage cleaning were applied – which then forces us to remove the decision-bearing portions of that summary before training. The resulting artefacts are the `*_timed_mistral` buckets used throughout the rest of the thesis.

Both single-label and multi-label variants are produced (`add_case_outcome_labels_mistral.py` and `add_multi_label_outcome_from_enriched.py`). This work uses the single-label variant as the prediction target; the multi-label variant is kept as a diagnostic artefact.

3.4.2 Coverage and Label Distribution

Labelling coverage and the resulting win/loss/neutral distribution are summarised in Figures 3.2 and 3.3. Coverage is near-complete across all five buckets; the class distribution is moderately imbalanced (roughly 60% WIN to 40% LOSS on the pooled cross-bucket dataset), which motivates the use of macro-F1 alongside accuracy as a primary metric in Chapter 6.

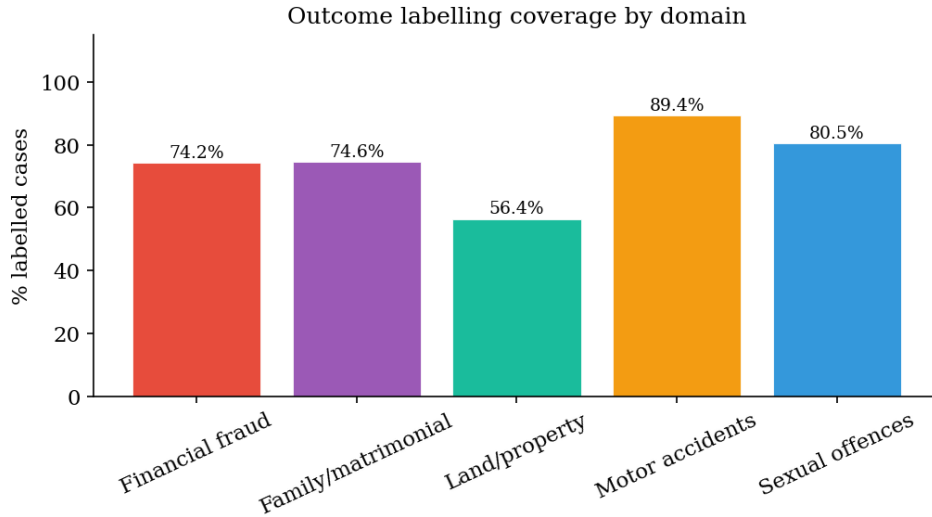


Figure 3.2: Mistral labelling coverage per bucket.

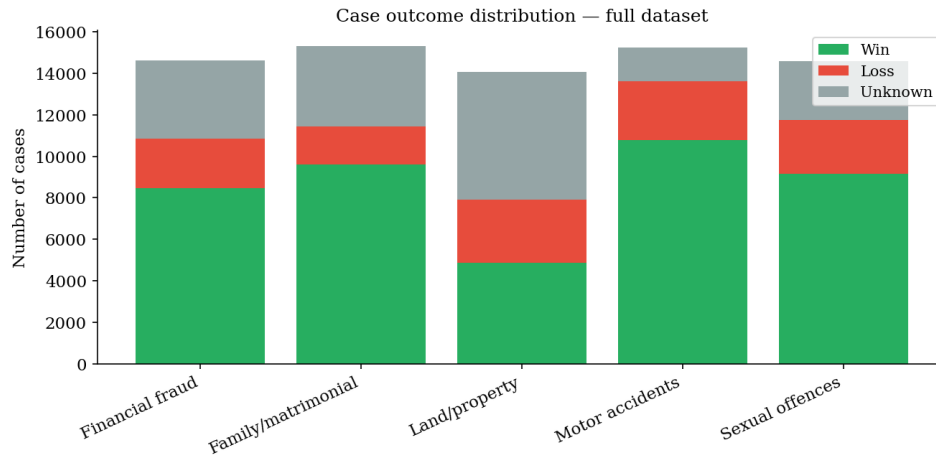


Figure 3.3: Outcome label distribution per bucket.

3.5 Multi-Hearing Merging

3.5.1 The Problem

A single legal matter frequently produces more than one judgment document: interim orders, remands, partial disposals, and the final decision may all be published as separate PDFs with distinct Indian-Kanoon-style filenames that nonetheless refer to the same case. If these hearing files are fed into the graph independently, three things go wrong:

1. duplicate CASE nodes for what is actually one dispute inflate the node count and double-count outcomes;
2. the outcome label of the final hearing is not propagated to the earlier hearings, so the training signal is noisy; and
3. cross-case structural signals (shared parties, statutes, and counsel across hearings) are double-counted as if they were independent observations.

3.5.2 Merge Strategy

The merging pipeline lives under `Thesis_FINAL_DATA/TIMELINE` and `DATA_SET_BUILDER_AND_EXPLORER/Timeline_Maker`. Per bucket, filenames are

parsed into a (`title`, `hearing_date`) tuple; the latest hearing becomes the *final* case, and every older hearing with the same title is *folded* into that final case’s JSON. The fold step either appends the older document’s text under a `prior_hearings` block (when the final document does not already contain that text) or is skipped when the final document already subsumes it. Every merge is logged in a per-bucket `report.json` so the operation is auditable and reversible.

The scale of the fold is non-trivial but concentrated:

Bucket	Inputs	Final cases	Folded in	Appended	Already contained
Family & Matrimonial	15,800	15,306	493	490	3
Financial Fraud	15,014	14,615	399	396	3
Land & Property	14,324	14,051	251	242	9
Motor Accidents	15,271	15,236	32	30	2
Sexual Offences	15,532	14,596	936	932	4
Total	75,941	73,804	2,111	2,090	21

Table 3.1: Multi-hearing merge statistics. 2,111 older hearing documents across the five buckets were folded into 2,090 merged final-case JSONs; the remaining 21 were already fully contained in the final hearing.

Sexual-offences matters fold most aggressively, which is consistent with their tendency to generate multiple remand and bail-stage orders before a final judgment; motor-accidents matters fold least, because the compensation process terminates in one tribunal order more often than it does in the other buckets.

3.6 Technology Stack

The pipeline is entirely Python-based. NER, RR, and summarisation run on top of `opennnyai` with `spaCy` 3.2.4 and CUDA-enabled `PyTorch` and `CuPy`, provisioned via the pinned `micromamba` environment `fixed_gpu_opennnyai_final`. Outcome labelling uses a `Mistral` model invoked through a batched client script. Multi-hearing merging is implemented with `merge_cases.py` and `merge_cases_v2.py` under the `Timeline_Maker` module. Graph construction is built on `torch_geometric`’s `HeteroData`, with text embeddings produced by the `BAAI/bge-m3` sentence-transformer

model running multi-process on GPU and cached per bucket. Training, ablation, and analysis scripts live under the `section_GNN` module.

Chapter 4

Heterogeneous Knowledge Graph Construction

4.1 Motivation: Why a Heterogeneous Graph?

The cleaned case JSONs produced by Chapter 3 are already richer than the raw PDFs: each record carries typed entities, rhetorical-role-segmented text, and an outcome label. The next question is how to represent that richness in a way that a learner can actually exploit.

A purely *flat* representation – concatenate the preamble, facts, and arguments of a case, feed the blob into a transformer, and classify – has two costs. The first is that it throws away the typed structure already present in the data: a court, a statute, a provision, and a party are four different kinds of object with four different kinds of evidential value for the outcome, and flattening them into one token stream makes the model re-learn that from scratch. The second, more important cost is that a flat representation has no way to exchange information *between* cases that share legally meaningful objects. When Case A and Case B both cite IPC 498A, the flat view sees the string twice; the graph view sees two cases attached to the same STATUTE node and can, through one round of message passing, let A’s representation influence B’s.

A heterogeneous graph captures both properties. It is *heterogeneous* because different node types carry different features and different relational semantics (a COURT is not a LAWYER and they should not share a projection matrix), and it is a *graph* because the legally important relationships are

relational: a case *is heard in* a court, an argument block *cites* a statute, a provision *belongs to* a statute, a lawyer *argues in* an argument block. These are exactly the edges a Graph Neural Network needs in order to reason about, rather than merely classify over, the textual content.

The rest of this chapter describes the schema that implements that intuition, the leakage-prevention strategy that keeps the schema from accidentally leaking the outcome, and the text-embedding and scalar features attached to each node.

4.2 Graph Schema

The graph is constructed as a **full-star global graph**: every case is a local "star" of text and entity nodes anchored on a CASE node, and a selected set of entity types are *shared* across cases so that stars of different cases are stitched into one connected component.

4.2.1 Node Types

The reasoning-focused graph keeps **17 node types** in three categories:

- **The case node.** A single CASE node per case, whose attributes store the outcome label, bucket, filing and decision dates (used for temporal gating), and bookkeeping metadata.
- **Text nodes.** Six types – PREAMBLE, FACTS, ARGUMENTS, PETITIONER_ARGUMENTS, RESPONDENT_ARGUMENTS, and OTHER_LAWYER_ARGUMENTS – each carrying a **bge-m3** text embedding of the cleaned, leakage-filtered span they represent (Section 4.4).
- **Entity nodes.** Ten types: PETITIONER, RESPONDENT, PETITIONER_LAWYER, DEFENCE_LAWYER, LAWYER, COURT, JUDGE, STATUTE, PROVISION, and PRECEDENT.

Four auxiliary NER types that OpenNyAI produces – ORG, GPE, DATE, and CASE_NUMBER – are explicitly *removed* from the reasoning-focused graph (FORBIDDEN_NODE_TYPES in the updated graph policy). They carry little predictive signal for the outcome and would otherwise inflate the node count and, worse, introduce weak, noisy edges that dilute message passing.

Their removal is one of the key differences between the reasoning-focused variant used in this thesis and the original baseline schema.

4.2.2 Cross-Case Sharing

Whether two cases share a node, or each hold a private copy of one, determines whether a GNN can use that node as a cross-case conduit. The policy (`reasoning_graph_policy.py`) is deliberate:

- **Shared across cases:** STATUTE, PROVISION, and PRECEDENT. These are the *law itself* – citing IPC 420 in one case and IPC 420 in another is meant to be the same object, and sharing the node is what lets the GNN propagate information along the citation.
- **Local to each case:** everything else, including COURT, JUDGE, and all LAWYER variants. These are *identity proxies*: if a particular judge or a particular bench has a historical win/loss bias, sharing the node would let the GNN recover that bias as a free shortcut to the outcome – exactly the kind of spurious signal the experimental design in Chapter 6 is built to prevent. Keeping them local makes the model reason about the law, not about the docket.

4.2.3 Edge Types

The schema defines a concrete set of typed edges. The core structural edges are:

- `case--has_preamble/has_facts/has_arguments--*text*`: attach each text node to its case.
- `case--has_petitioner_arguments/has_respondent_arguments/has_other_lawyer_arguments--*text*`: attach party-specific argument blocks.
- `case--has_petitioner/has_respondent--*party*, case--heard_in--court, case--decided_by_bench--judge, case--has_petitioner_lawyer/has_defence_lawyer/has_lawyer--*counsel*`.

- `arguments--cites_statute--statute`, `arguments--cites_provision--provision`, `arguments--cites_precedent--precedent`: the citation backbone, attached to the ARGUMENTS node rather than to the case so that the citation sits in its legal-discourse context.
- `provision--belongs_to_statute--statute`: stitches the provision hierarchy into the graph so that a citation to Section 482 is two hops from IPC, not disconnected from it.
- `petitioner_lawyer--citation--petitioner_arguments`, `defence_lawyer--citation--respondent_arguments`, `lawyer--citation--other_lawyer_arguments`: connect each counsel to the argument block they are responsible for.
- `petitioner--is_party_in_arguments--petitioner_arguments`, `respondent--is_party_in_arguments--respondent_arguments`: tie parties to their own argument blocks.

All edges are added with their reverse counterparts (`include_reverse_edges`), and the final graph is passed through `torch_geometric.to_undirected` so that HGT’s typed message passing can flow in both directions.

Shortcut edges that are intentionally removed. An earlier version of the schema added edges that let entities bypass the arguments node – e.g. `statute-used_in_arguments-arguments`, `judge-presided_arguments-arguments`, `petitioner/respondent-is_party_in_arguments-arguments` (to the *shared*, un-partitioned ARGUMENTS node). The updated policy *forbids* these edges (`FORBIDDEN_EDGE_TYPES`). Keeping them would have created short paths from identity proxies to the case representation that contribute little reasoning value but a lot of spurious correlation; the thesis uses the reasoning-focused schema for exactly this reason.

4.2.4 Attributes

Every CASE node carries a binary outcome label (+1 win, -1 loss) plus its bucket membership and the dates needed for temporal gating. Every text

node carries the raw text (for downstream inspection only; the model never sees it directly) and the pre-computed embedding. Every entity node carries its canonical name and the canonicalisation metadata needed to reproduce the merge. Edges are attributed with their relation type, which is what the HGT layers use to index their per-relation parameters (Chapter 6).

4.2.5 Visualizing the Graph Structure

To provide a concrete visual intuition of the graph schema described above, we present diagrams of a single case graph and a multi-case subgraph.

Figure 4.1 illustrates the local “star-graph” structure surrounding a single CASE node. The node serves as an anchor connected to its text components (e.g., Preamble, Facts, Arguments) and its identity entities (e.g., Court, Judge, Lawyers). From the Arguments node, further citation edges branch out to Statutes and Precedents.

Importantly, these outward citation edges maintain a strict chronological hierarchy: a newer case will only form a citation edge to a Precedent if $\text{precedent_year} < \text{case_year}$, and to a Statute if $\text{statute_year} \leq \text{case_year}$. This temporally-directed structure explicitly ensures that older cases cannot structurally link forward in time to newer artifacts.

Figure 4.2 demonstrates the cross-case sharing mechanism. To prevent spurious correlation (docket bias leakage), nodes like Courts and Judges are kept private to each case. In contrast, legal entities such as Statutes, Provisions, and Precedents are shared across cases, forming the cross-case conduits that allow the GNN to perform structural and legal reasoning without directly observing the outcome proxy.

4.3 Leakage Prevention Strategy

Legal judgment prediction has a notorious failure mode: models that appear to predict the outcome are actually *reading* it. The construction pipeline defends against this at three distinct points.

1. **Field-level removal.** The cleaning stage drops a fixed set of `FORBIDDEN_TOP_LEVEL_FIELDS` from the case JSON before any graph is built: `case_outcome_label`, `case_outcome_score`, `llm_case_outcome`, `decision_text`, `rpc_texts`, `short_explanation`, and

Sample star-graph structure for a single Case node

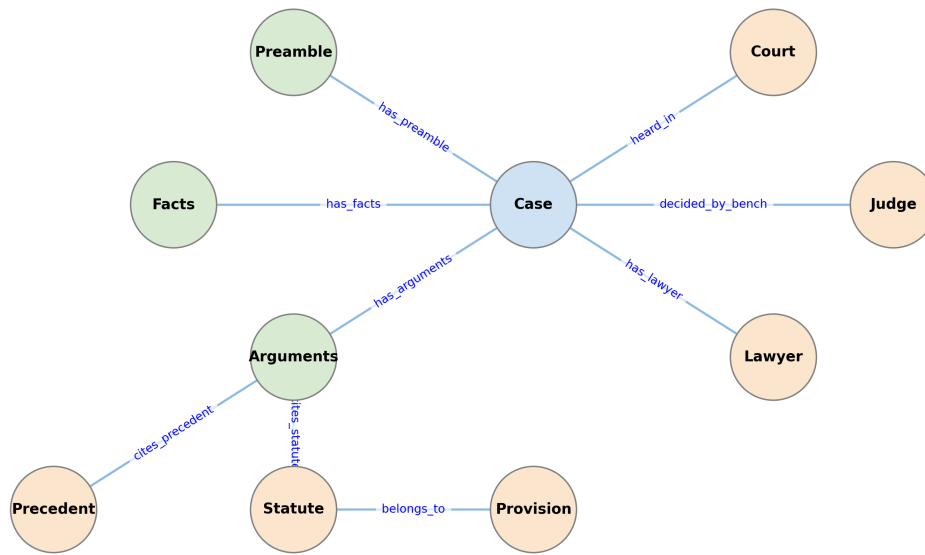


Figure 4.1: Sample star-graph structure for a single Case node. Various text blocks feature as text nodes, while details like Court and Judge form local entity nodes. The Arguments node branches out to cite shared legal concepts like Statutes and Precedents.

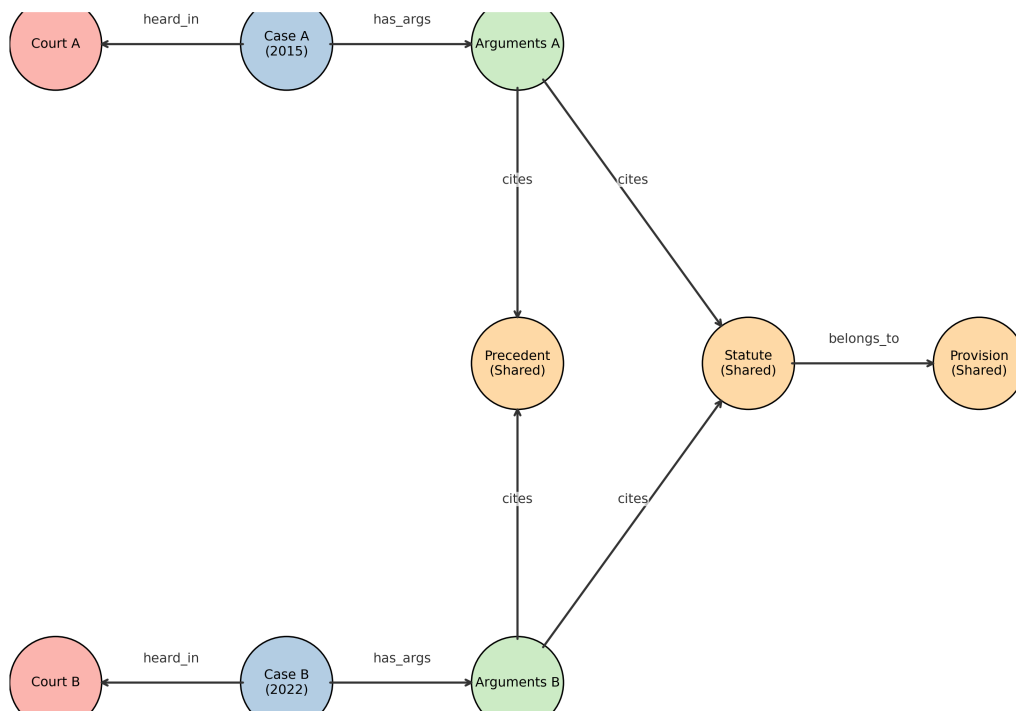


Figure 4.2: Cross-connection of cases via shared nodes. Each case maintains its own local representations for identity entities to prevent shortcut leakage. Shared nodes like Statutes and Precedents act as legally-sound cross-case conduits for message passing.

`raw_model_response`. It also drops any `raw_result.summary.decision` block produced by the OpenNyAI summariser.

2. Rhetorical-role filtering. The `FORBIDDEN_ANNOTATION_LABELS` set (`{RPC, RLC}`) plus a label-string blacklist (`decision`, `disposition`, `operative`, `order`, `rpc`) causes any annotation in those classes to be removed from the summary before the text nodes are built. Because `RPC` is where the court actually pronounces the result, this is the single most important filter: without it, the `ARGUMENTS` text node would contain the verdict verbatim.

3. Phrase-level masking. A list of `DEFAULT_LEAKAGE_PHRASES` – “*petition stands disposed of*”, “*appeal is allowed*”, “*set aside*”, “*quashed*”, and so on – is compiled into case-insensitive, whitespace-tolerant regex patterns and applied as a sub-sentence scrubber across every retained text field. Every match is logged in a per-case `leakage_audit`, and the phrase itself is replaced with a `[LEAKAGE_MASK]` token. This catches residual decision-bearing fragments that the `RR` filter missed because they were embedded inside a `FACTS` or `ARGUMENTS` sentence rather than tagged as `RPC`.

4. Temporal gating on shared entities. Sharing a `PRECEDENT` or `STATUTE` node across cases would leak future information if a precedent decided in 2022 were attached to a case from 2015. The graph builder therefore enforces a temporal rule inside `case_star_builder`: a precedent citation edge is added only when `precedent_year < case_year`, and a statute citation edge only when `statute_year ≤ case_year`. This keeps the cross-case conduit legally honest.

5. Audit trail. Every case carries a `leakage_audit` structure recording the fields dropped, the annotations removed, the decision text removed, and every phrase-level mask. The audit is kept with the graph cache so that any future claim of leakage can be *disproved*, not merely argued against.

The combined effect of these five gates is that the text a GNN layer eventually sees through its text-node features is a pre-decision representation of the dispute – who the parties are, what happened, what each side argued, and what law they cited – with the decision itself systematically withheld.

4.4 Text Embedding and Feature Extraction

4.4.1 The bge-m3 Text Encoder

All text nodes are featurised with BAAI/bge-m3, a multilingual sentence-transformer with a 1024-dimensional output, run through the `sentence_transformers` library. The choice of bge-m3 is driven by two concrete requirements: (i) Indian legal text mixes English with Hindi and regional-language transliterations, Latin-script legal abbreviations, and citation fragments, and bge-m3’s multilingual training data handles this noticeably better than English-only encoders; and (ii) the model is strong in retrieval settings, which aligns with the graph’s use of embeddings as comparison features across cases.

Configuration. The encoder is run with `batch_size=16`, `max_length=512`, on GPU, and with multi-process mode enabled. When multiple CUDA devices are visible the encoder spins up one worker per device, distributes the text list, and merges the results, which is what makes encoding millions of argument and facts spans tractable in wall-clock terms.

Hierarchical chunking for long spans. Legal judgment *Facts* and *Arguments* sections routinely exceed the model’s 512-token window. For texts longer than the configured character limit, the encoder splits them into overlapping, sentence-boundary-aligned chunks, encodes each chunk independently, and mean-pools the resulting vectors. This preserves a consistent 1024-dimensional representation per text node regardless of input length, while avoiding the information loss of a hard truncation.

Caching. Embeddings are cached per namespace with a deterministic hash of the input ids, texts, and encoder configuration. Re-running the graph build with an unchanged encoder loads cached vectors in seconds; changing any encoder-sensitive field (`backend`, `model_name`, `max_length`, `hierarchical_chunk_chars`) invalidates the cache. A `hashing` fallback encoder is available for local debugging when GPU resources are unavailable.

4.4.2 Node Feature Assembly

For each node type the final per-node feature vector is the concatenation of:

- the 1024-dim `bge-m3` embedding of the node’s canonical text (for text nodes, that is the full span; for entity nodes, that is the canonical name);
- scalar node-level features: local frequency in the case, section-of-first-mention indicators (`seen_in_preamble`, `seen_in_arguments`), type one-hot, and for shared entity types the log global frequency across the corpus;
- for the CASE node, bucket one-hot and the outcome label is attached only as the supervision target, never as a feature.

Because different node types carry different feature dimensionalities, the HGT model used in Chapter 6 begins with a per-node-type linear projection (`input_projections`) that maps all features into a common hidden dimension before the first heterogeneous message-passing layer. This is the mechanism that lets the graph remain genuinely heterogeneous at the feature level while still admitting a uniform message-passing computation.

Chapter 5

Graph Exploration and Structural Insights

5.1 Why Explore the Graph Before Prediction?

It is tempting to treat the graph built in Chapter 4 as a black box that is fed into a GNN and evaluated by a single F1 number. That view is wrong for two reasons.

First, *the graph is the dataset*. In a flat-text pipeline the dataset is a set of labelled documents; here the dataset is a typed, relational object with cross-domain structure, and any claim a later chapter makes about the model’s behaviour presupposes a concrete understanding of that structure. Whether the model’s accuracy is driven by genuine cross-case reasoning or by a few dominant hub statutes, whether the five legal domains are actually five or mostly one, whether the per-case subgraph has the connectivity a message-passing layer needs – none of these questions can be answered by looking at prediction results alone.

Second, *leakage-safety claims are structural claims*. The leakage prevention strategy in Section 4.3 is only as convincing as the structural evidence that the remaining graph is still dense enough to support learning. If removing RPC annotations, decision fields, and identity-proxy shortcuts left behind a near-disconnected graph, the downstream accuracy would be either implausibly high (still leaking) or implausibly low (now uninformative). The exploration stage is where we verify that neither is the case.

This chapter therefore treats the graph as an object of study in its own

right. Section 5.2 summarises the overall structure; Section 5.3 breaks it down per legal domain; Section 5.4 distils the conclusions that feed forward into Chapter 6.

5.2 Graph Statistics and Structural Analysis

5.2.1 Overall Graph Statistics

The final reasoning-focused graph, assembled across all five legal domain buckets, contains **880,176 nodes** and **1,977,295 edges**. The average node degree is **4.49**; the maximum degree is **29,602**, attained by the IPC statute node.

At the per-bucket level (before cross-bucket sharing is expanded into the unified view the GNN trains on), the five buckets have broadly comparable case counts but markedly different internal densities:

Bucket	Cases (files)	Nodes	Edges
Family & Matrimonial	15,307	97,150	3,660,068
Financial Fraud	14,616	113,240	7,869,952
Land & Property	14,052	126,962	9,210,612
Motor Accidents	15,237	125,234	7,165,461
Sexual Offences	14,597	106,662	7,123,225
Cross-bucket unified	73,809	485,540	27,243,141

Table 5.1: Per-bucket and cross-bucket graph statistics. The cross-bucket graph is denser than the sum of the per-bucket graphs because shared STATUTE, PROVISION, and PRECEDENT nodes acquire edges from every bucket that cites them.

Family-matrimonial is the sparsest bucket by edges-per-case (≈ 239), and land-property is the densest (≈ 655); the difference is driven mostly by the number of distinct provisions and precedents cited per case in each domain.

5.2.2 Node Type Distribution

The node-type breakdown of the unified graph is strongly skewed towards **parties and precedents**:

Node type	Count	Share
CASE	72,735	8.3%
PRECEDENT	255,148	29.0%
RESPONDENT	192,147	21.8%
PETITIONER	171,084	19.4%
LAWYER	73,645	8.4%
PROVISION	43,504	4.9%
COURT	38,799	4.4%
STATUTE	21,754	2.5%
JUDGE	11,360	1.3%

Table 5.2: Distribution of the 880,176 nodes by type in the unified reasoning-focused graph.

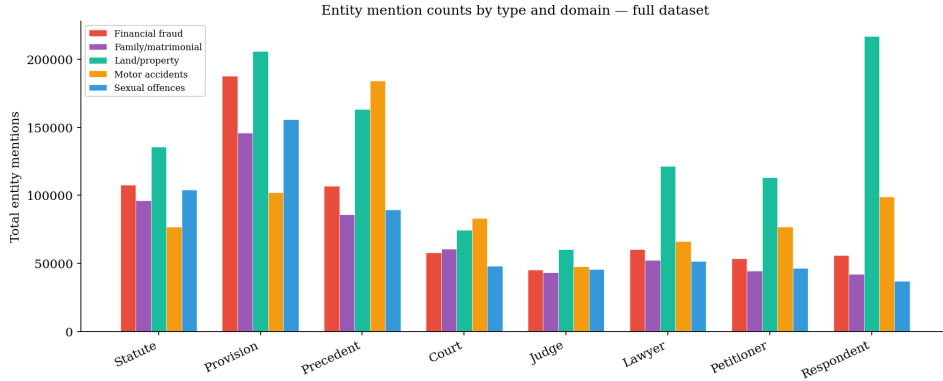


Figure 5.1: Entity type totals across the corpus.

Two observations follow. First, precedents outnumber every other non-party type by an order of magnitude – citation is by far the dominant legal artefact in the graph, which is why precedent sharing is a load-bearing edge type. Second, parties (PETITIONER + RESPONDENT) account for more than 40% of nodes; because they are held *local* to each case rather than shared, they contribute many nodes but no cross-case edges. The statute side of the graph is small in node count but, as the next subsection shows, carries disproportionate structural weight.

5.2.3 Degree Distribution and Hub Entities

The graph is strongly hub-dominated. The top-12 nodes by degree are all statutes, provisions, or courts – exactly the node types the schema shares across cases:

Node	Type	Degree	Buckets bridged
IPC	Statute	29,602	5
CR.P.C.	Statute	18,704	5
Supreme Court	Court	17,294	5
Apex Court	Court	14,242	5
Indian Penal Code	Statute	12,009	5
High Court of Judicature at Allahabad	Court	8,833	5
Section 482	Provision	8,732	5
Section 506	Provision	8,258	5
I.P.C.	Statute	7,051	5
Section 420	Provision	6,285	5
Section 323	Provision	6,002	5
Constitution of India	Statute	5,950	5

Table 5.3: Top hub entities by degree, with the number of buckets they bridge. Every one of the top 12 hubs appears in all five legal domains.

The PageRank ranking tells the same story from a different angle: IPC (PR = 0.0114), Supreme Court (0.0082), CR.P.C. (0.0069), and Apex Court (0.0067) dominate the top of the distribution. That every one of the top hubs bridges all five buckets is not an accident of the numbers – these are the statutes and courts every Indian criminal and civil proceeding touches – but it has a concrete modelling consequence: these hubs are where cross-case message passing actually happens.

The corollary is that the graph has *many* near-isolated nodes too. Most precedents appear in a single case; most parties appear in one matter; most judges hear a bounded docket. This two-regime structure – a small set of super-hubs plus a long tail of single-case leaves – is what the “connectivity score” (Section 5.2.6) is designed to quantify.

5.2.4 Cross-Domain Bridge Analysis

The buckets are not isolated. Table 5.3 already shows that the top hubs are bridges by construction; the full bridge analysis confirms that the connective tissue between domains is dominated by the general criminal-procedure and penal-code statutes, with domain-specific bridges layered on top:

- IPC, CR.P.C., INDIAN PENAL CODE, CONSTITUTION OF INDIA, and SUPREME COURT / APEX COURT appear in all five buckets and together account for the majority of cross-bucket edges.
- Domain-specific statutes bridge fewer buckets but at higher within-bucket intensity: POCSO ACT appears in four buckets but is heavily concentrated in sexual-offences; MOTOR VEHICLES ACT, 1988 appears in all five but is concentrated in motor-accidents; LAND ACQUISITION ACT, 1894 is concentrated in land-property.
- Provisions like Section 482, 506, 420, and 323 are multi-bucket because they sit in the CrPC and IPC, which are themselves multi-domain.

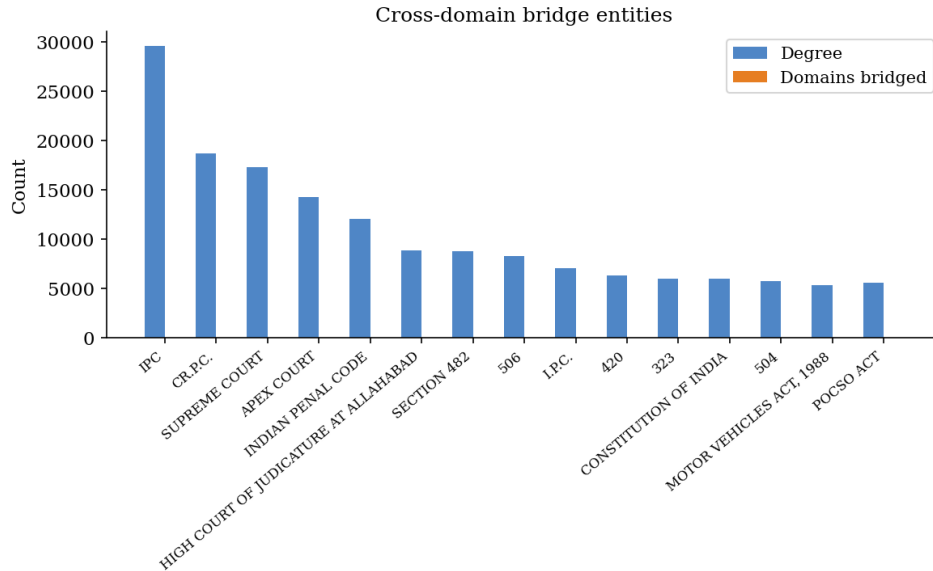


Figure 5.2: Cross-bucket bridges: entities that connect more than one legal domain, ranked by the number of buckets they appear in.

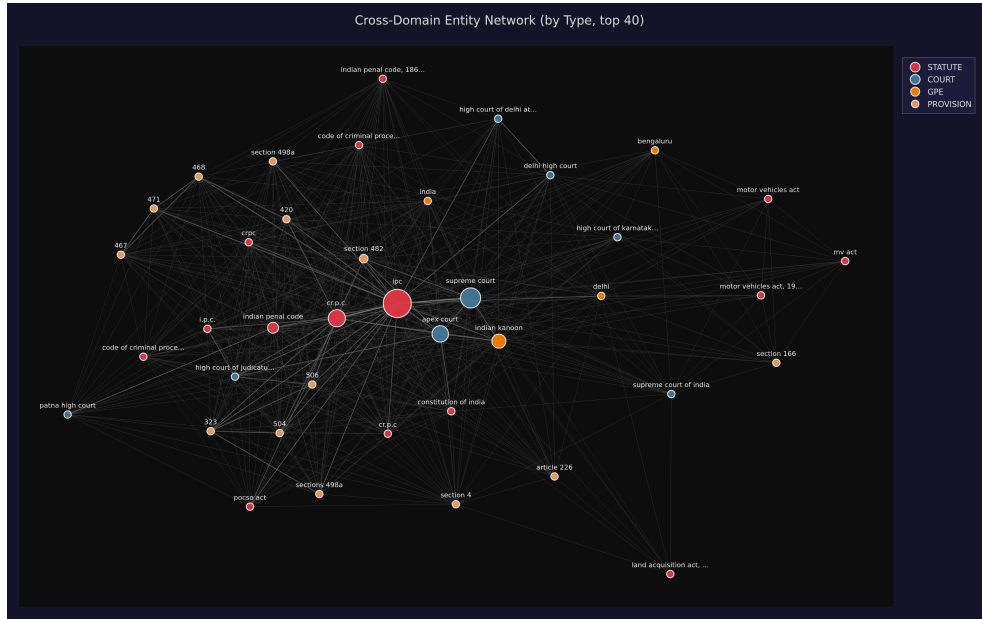


Figure 5.3: Cross-domain entity network for the highest-PageRank shared entities, coloured by node type. Unlike the bridge-count bar chart, this view shows the *shape* of the shared legal spine: which entities form the dense centre, and which remain peripheral even when they appear across multiple domains.

Figure 5.3 adds a topological point that the aggregate bridge counts alone do not make visible. The cross-domain backbone is narrow and authority-driven: IPC, CR.P.C., SUPREME COURT, and APEX COURT occupy the centre, while bucket-specific statutes such as the MOTOR VEHICLES ACT, POCSO ACT, and LAND ACQUISITION ACT attach as secondary clusters rather than co-equal hubs. A further visual detail is that GPE nodes (*Indian Kanoon*, Delhi, Bengaluru) sit mostly on the outside of the network. They co-occur often enough to be visible, but they do not organise the graph the way statutes, provisions, and courts do. This sharpens the interpretation of cross-domain transfer: message passing is not occurring through a uniformly shared legal space, but through a small common authority backbone that then branches into bucket-specific statutory neighbourhoods.

5.2.5 Entity Sharing Heatmap Across Domains

The entity-sharing heatmap (Figure 5.4) visualises, for each pair of buckets, the number of shared canonical entities. Three patterns are visible:

1. a strong sharing between *Family & Matrimonial*, *Financial Fraud*, and *Sexual Offences*, all of which are IPC/CrPC-driven;
2. moderate sharing between those three and *Motor Accidents*, mediated by the procedural statutes and by the Supreme Court / High Court nodes;
3. comparatively weaker sharing with *Land & Property*, which has its own statutory backbone (Land Acquisition Act, Constitution).

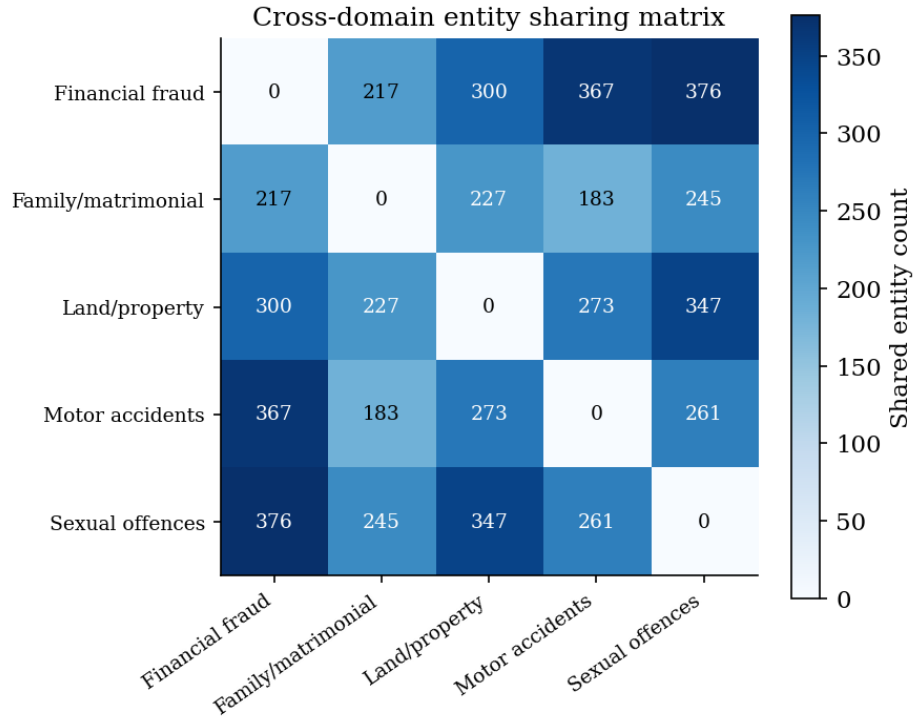


Figure 5.4: Entity sharing heatmap across the five legal domains.

This pattern is exactly what a cross-domain transfer claim needs: the domains are related enough that a shared representation should carry useful

signal, but different enough that a single-domain classifier should not dominate a cross-domain one unless that shared signal is actually being used.

5.2.6 Connectivity Score Distribution

A **connectivity score** is assigned to each case as a bucket-normalised measure of how well that case is wired into the rest of the graph through its shared-entity neighbours (statutes, provisions, precedents). High-connectivity cases cite hub statutes and widely-cited precedents; low-connectivity cases sit on rare or single-case entities.

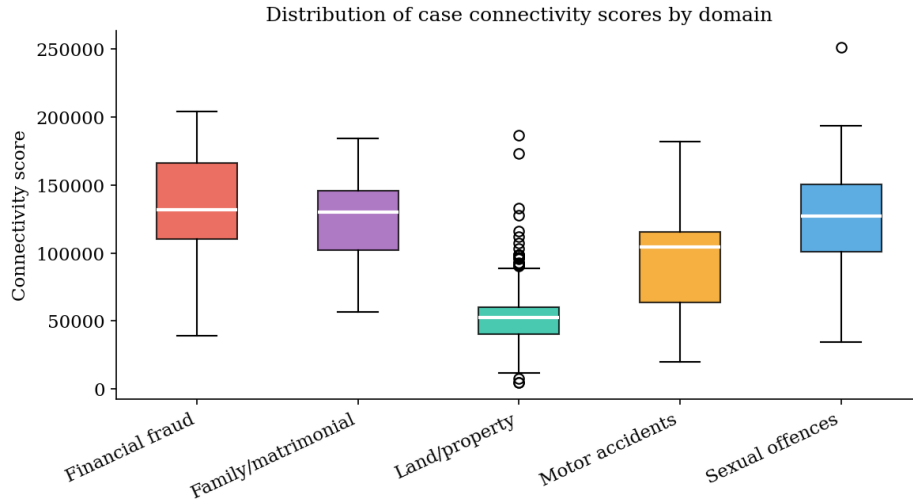


Figure 5.5: Connectivity score distribution per bucket.

The score is used by the Graph Visualiser (`GRAPH_VISUALISER/`) to rank cases when sampling Hub-Network and Case-Connection views, and is a useful diagnostic for message passing: a GNN layer’s information gain on a low-connectivity case is structurally bounded, and the ablation pattern in Chapter 6 consistent with this expectation.

5.3 Per-Domain Corpus Analysis

The structural view above aggregates across buckets; the per-domain view is where the legal character of each bucket shows up.

5.3.1 Case Length and Sentence Distributions

Case length varies substantially across buckets. Motor-accidents judgments are typically short and tribunal-style; land-property matters are long and document-heavy (deeds, surveys, and possession history); family-matrimonial sits in the middle, dominated by petition-style fact sections. Sentence-count distributions follow the same ordering and are the primary driver of the hierarchical-chunking configuration in Section 4.4: long-tail FACTS and ARGUMENTS sections in land-property and financial-fraud buckets exceed the bge-m3 context window and are mean-pooled across overlapping chunks.

5.3.2 Entity Richness Per Domain

Entity richness – unique canonicalised entities per case – is highest in land-property and financial-fraud, which carry many distinct parties, precedents, and provisions per matter, and is lowest in motor-accidents, which repeatedly cites the same small set of provisions (Section 166, Motor Vehicles Act). The PageRank ranking within each bucket makes this concrete:

- **Family & Matrimonial:** IPC, CrPC, Sections 498A, 482, 498A.
- **Financial Fraud:** IPC, CrPC, Sections 420, 468, 471.
- **Land & Property:** Supreme Court, Land Acquisition Act 1894, *Indian Kanoon* (a source GPE), Constitution of India.
- **Motor Accidents:** Supreme Court, Motor Vehicles Act 1988, Apex Court, Section 166, Motor Vehicles Act.
- **Sexual Offences:** IPC, CrPC, POCSO Act, Indian Penal Code, Section 506.

The top-5 per bucket is a near-perfect fingerprint of the underlying legal domain, which is a reassuring sanity check: the canonicalisation and the NER are recovering legally meaningful primary citations, not surface noise.

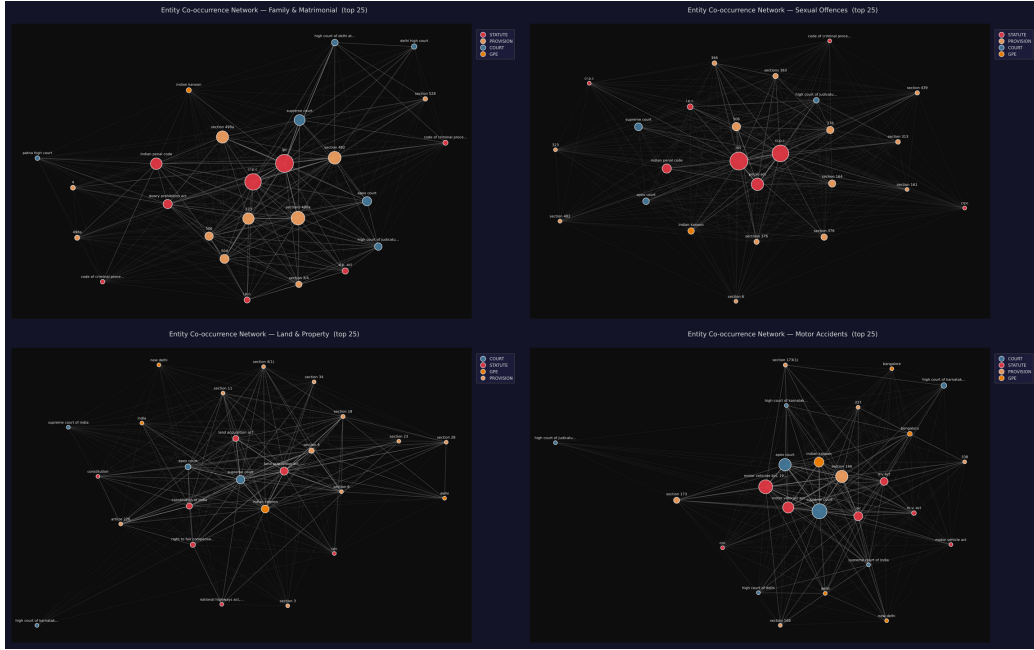


Figure 5.6: Representative within-bucket entity co-occurrence networks for four legal domains, each restricted to the top-25 PageRank entities in that bucket. The figure is intended as a structural fingerprint view: not just which entities are important, but how each bucket’s important entities organise into a distinct local backbone.

Figure 5.6 shows that the domains differ not only in their top entities but in their *network geometry*. Family & matrimonial and sexual offences both remain visibly IPC/CRPC-centred, yet they split into different provision clusters almost immediately: Section 498A / 482 and dowry-law citations in the former, versus POCSO / Section 376 / 363 clusters in the latter. Land-property is more spatially spread, with several semi-central statutory and constitutional nodes rather than one overwhelmingly dominant criminal-procedure core; this is consistent with its higher entity richness and document-heavy reasoning style. Motor-accidents, by contrast, exhibits the tightest and most repetitive local backbone around the MOTOR VEHICLES ACT and Section 166, suggesting that many cases enter the graph through a comparatively narrow set of recurring authorities. Financial-fraud is omitted here for space, but its readable network shows the same kind of provision-

centred motif around Sections 420, 468, and 471. Together these bucket-level snapshots make the earlier “domain fingerprint” claim visual rather than purely tabular.

5.3.3 Rhetorical Role Composition Per Domain

The rhetorical-role composition of each bucket – once RPC/RLC are removed by the leakage filter – is dominated by FACTS and ARGUMENTS, with a stable PREAMBLE share across domains. The relative size of the ARGUMENTS block tracks case length: it is largest in land-property and financial-fraud and smallest in motor-accidents.

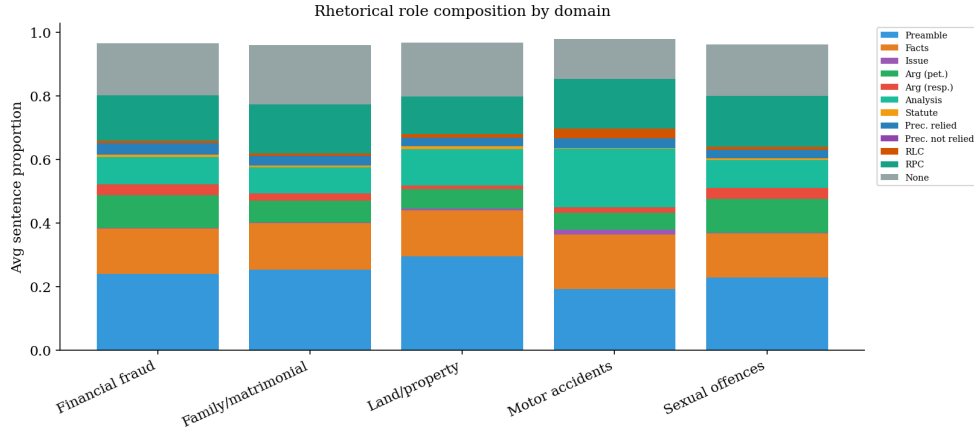


Figure 5.7: Rhetorical role breakdown per bucket after leakage-safe filtering. RPC and RLC annotations are intentionally absent.

5.3.4 Outcome Distribution Per Domain

The outcome label distribution (Figure 3.3) is moderately imbalanced and bucket-dependent. Motor-accidents is the most WIN-skewed, consistent with compensation claims being allowed more often than rejected at the appellate stage; financial-fraud and sexual-offences are closer to balanced. The cross-bucket pooled distribution is roughly $\sim 60\%$ WIN to $\sim 40\%$ LOSS, which is why Chapter 6 reports macro-F1 alongside accuracy and ROC-AUC.

5.4 Key Structural Insights

Five insights carry forward into the prediction chapter:

1. **The graph is hub-dominated.** A small number of cross-bucket statutes and courts (IPC, CrPC, Supreme Court) are the spine of the graph; most other entities are long-tail leaves. Any claim the model makes about “cross-case reasoning” must be interpretable against this topology.
2. **Cross-domain sharing is real but uneven.** The four IPC/CrPC-driven buckets share heavily with each other; land-property partially detaches via its own statutory backbone. A cross-domain model should outperform a naive concatenation only if it can actually exploit the shared spine.
3. **Identity-proxy leakage is structural.** Judges, specific lawyers, and specific courts have bridge and betweenness scores high enough to make *sharing* them across cases unsafe; the decision to hold them local is vindicated by the bridge analysis.
4. **Connectivity score predicts modelling difficulty.** Low-connectivity cases have, by construction, fewer shared-entity neighbours to aggregate from; the per-domain accuracy pattern in Chapter 6 is consistent with connectivity being a first-order driver of case-level difficulty.
5. **The per-domain fingerprints are legally coherent.** Each bucket’s top-PageRank entities are exactly what a lawyer would expect; this is the single strongest evidence that the pipeline’s NER, canonicalisation, and rhetorical-role filtering together produce a legally faithful, not merely statistically clean, corpus.

Chapter 6

Prediction Model and Results

6.1 Why a Heterogeneous Graph Transformer?

The exploration in Chapter 5 made two demands of the model. First, the graph has *many node types* and each carries its own semantics – a STATUTE is not a CASE, and message-passing with a shared weight matrix would force the two to live in the same geometry. Second, the graph has *many edge types* and the structural role of an edge depends on its type: `cites_statute`, `heard_in`, and `belongs_to_statute` should not share aggregation weights.

The Heterogeneous Graph Transformer (HGT) satisfies both requirements. It maintains per-node-type projections, per-edge-type messages, and multi-head attention over the heterogeneous neighbourhood, which is what a typed message-passing layer needs to reason about the graph as the relational object it is rather than as a flat homogeneous approximation. Compared to the two natural alternatives – RGCN and HAN – HGT additionally learns the relative importance of different relations via attention rather than via a fixed per-relation weight, which turns out to matter on this graph because the predictive weight of (say) `arguments` $\rightarrow$ `cites_statute` $\rightarrow$ `statute` is not the same as `case` $\rightarrow$ `heard_in` $\rightarrow$ `court`.

6.2 Model Architecture

Input projection. Every node type is first mapped to a common hidden dimension by a per-type linear layer (`input_projections`), and a learnable type embedding (`type_embeddings`, initialised at $\mathcal{N}(0, 0.02^2)$) is added. This

preserves the heterogeneity of the input features (1024-dim `bge-m3` for text nodes, concatenated scalar features for entity nodes) while giving the message-passing layers a uniform geometry to work in.

Stacked HGT layers. Two HGTConv layers with four attention heads and hidden dimension 128 form the message-passing core. Each layer is wrapped in a residual connection with per-node-type `LayerNorm`, dropout ($p = 0.2$), and a ReLU non-linearity. Two layers is deliberate: the graph diameter between a CASE and a shared STATUTE is two hops (case $\rightarrow$ arguments $\rightarrow$ statute), so two layers is sufficient to move information between the two kinds of signal the model needs – the case’s own textual evidence and the cross-case statutory neighbourhood.

Classifier head. The CASE node’s final hidden representation is fed through a two-layer MLPHead producing the binary outcome logits.

Reference implementation. The architecture is parametric in `hidden_dim`, `num_layers`, `num_heads`, and `dropout`, and also supports a HeteroConv/SAGE fallback for ablation purposes; all experiments in this chapter use the HGT configuration above unless stated otherwise.

Training configuration. AdamW with learning rate 1×10^{-3} and weight decay 1×10^{-5} , cross-entropy loss with balanced class weights, 60 epochs with best-validation-F1 early stopping (patience 10). The loss and optimiser run on the full transductive graph; only the case-node indices are masked into train/val/test partitions. The `party_args_lr_decay` variant additionally applies a layer-wise learning-rate decay to the party-and-argument sub-module, which, as Section 6.6 shows, is the best-performing configuration.

6.3 Experimental Setup

Transductive protocol. Training is *transductive*: the full graph – including the test cases’ text nodes and entity neighbourhoods – is visible to the model, but the test cases’ *labels* are withheld through a case-node mask. This matches the real-world use case (a new judgment arrives and its graph is built using the same canonical entities as the training cases) and it is what allows shared hub nodes like IPC to carry information across the split.

5-fold cross-validation. Cases are partitioned into 5 stratified folds (seeds 42–46). For each fold: 72% train, 8% validation, 20% test. The validation fold drives early stopping; the test fold is evaluated once per fold with the best validation checkpoint. Reported numbers are the mean and sample standard deviation across the 5 folds.

Evaluation settings. Six evaluation settings are produced per experiment: one *cross-bucket* setting that pools all five legal domains into a single 72k-case dataset, and five *per-bucket* settings that restrict the graph and the labels to a single domain. The per-bucket runs share the same architecture and training configuration; only the graph and the cases differ.

Metrics. Primary: macro-F1 (robust to the mild outcome imbalance). Secondary: accuracy, ROC-AUC. Per-class precision, recall, F1, and support are reported in the supplementary JSON logs but omitted from the text for brevity.

6.4 Ablation Variants and Rationale

Seven ablations, each a targeted structural or featural removal from the baseline:

- **case_node_minimised** – strip the CASE node of everything except the minimum required for classification, so that all case-specific information has to flow through text and entity neighbours.
- **no_names** – remove party, lawyer, and judge name content from node features; keeps structural connectivity.
- **no_cross_case** – disable sharing of STATUTE, PROVISION, and PRECEDENT across cases; every case is an isolated star.
- **text_only** – remove entity nodes entirely; the graph becomes a case-plus-text-only hierarchy.
- **hierarchical_enc** – enable hierarchical chunked encoding for all text nodes (long arguments are mean-pooled across overlapping chunks rather than hard-truncated).

- `section_sep_enc` – encode each rhetorical-role section with a distinct encoder head so their features live in disjoint subspaces.
- `party_args_lr_decay` – apply layer-wise learning rate decay to the party-and-argument sub-module so those sub-streams learn slower and remain more faithful to their pre-trained initialisations.

6.5 Baseline Comparisons

Before comparing the graph ablations against one another, it is useful to anchor the task with the strongest non-graph reference models and the unmodified HGT baseline.

Configuration	Setting	Accuracy	Macro-F1
Majority-class baseline	Fold-0 reference	0.609	0.378
Raw-feature logistic regression	Fold-0 reference	0.744	0.736
HGT baseline	5-fold cross-bucket mean	0.7854 ± 0.0013	0.7785 ± 0.0013

Table 6.1: Reference baselines for the pooled cross-bucket task. The first two rows come from the fold-0 probing package and are included to anchor task difficulty; the HGT row is the full 5-fold cross-bucket baseline used throughout the main result tables.

This baseline view clarifies the scale of the modelling problem before the ablation tables are introduced. A plain majority predictor is inadequate, and even a strong non-graph reference built on the raw pre-GNN case features reaches only 0.744 accuracy. The full HGT baseline improves that by roughly four points in accuracy and just over four points in macro-F1, which is enough to justify treating the heterogeneous graph as more than a decorative representation.

6.6 Cross-Bucket Results

Pooling the five legal domains into a single transductive graph yields the following cross-bucket results ($N=72,735$ cases, 5-fold CV):

Configuration	Accuracy	Macro-F1	ROC-AUC
HGT baseline	0.7854 ± 0.0013	0.7785 ± 0.0013	0.8694
+ <code>section_sep_enc</code>	0.7919 ± 0.0084	0.7836 ± 0.0071	0.8729
+ <code>text_only</code>	0.7918 ± 0.0044	0.7843 ± 0.0035	0.8743
+ <code>no_names</code>	0.7874 ± 0.0046	0.7806 ± 0.0044	0.8694
+ <code>no_cross_case</code>	0.7857 ± 0.0021	0.7782 ± 0.0013	0.8682
+ <code>hierarchical_enc</code>	0.7828 ± 0.0046	0.7771 ± 0.0033	0.8701
+ <code>case_node_minimised</code>	0.7772 ± 0.0066	0.7706 ± 0.0059	0.8588
+ <code>party_args_lr_decay</code>	0.8028 ± 0.0053	0.7955 ± 0.0053	0.8847

Table 6.2: Cross-bucket 5-fold CV results. The HGT baseline already improves on the non-graph references in Table 6.1; the best configuration applies layer-wise LR decay to the party-and-argument sub-module.

Three observations anchor the rest of the chapter.

The pooled winner is clear. The `party_args_lr_decay` configuration is the strongest cross-bucket model at 0.8028 accuracy and 0.7955 macro-F1, a non-trivial improvement over the plain HGT baseline. The gain is not dramatic, but it is consistent across folds and large enough to matter on a task where the baseline itself is already strong.

Cross-case sharing is doing something, but less than expected. The `no_cross_case` configuration, which disables sharing of STATUTE, PROVISION, and PRECEDENT across cases, scores within noise of the baseline (0.7857 vs. 0.7854). That does not mean cross-case edges are useless – the hub structure of Chapter 5 still shows they carry genuine signal – but it does mean that in the transductive pooled regime, the text features are strong enough that removing explicit cross-case paths is largely compensated by the remaining within-case structure. This is the single most important finding of the ablation and is discussed further in Section 6.8.

Identity removal does not hurt. The `no_names` configuration, which strips party, lawyer, and judge *names* from the node features, scores 0.7874 – slightly *above* the baseline. The model is not relying on party identity to predict the outcome, which is the operational answer to the identity-proxy leakage concern raised in Section 4.3.

6.7 Per-Domain Results

Bucket	Accuracy	Macro-F1	ROC-AUC
Family & Matrimonial	0.8155 ± 0.0099	0.7978 ± 0.0060	0.8934
Financial Fraud	0.7766 ± 0.0086	0.7639 ± 0.0085	0.8456
Land & Property	0.8156 ± 0.0054	0.7953 ± 0.0069	0.8759
Motor Accidents	0.8301 ± 0.0045	0.7944 ± 0.0072	0.8904
Sexual Offences	0.7711 ± 0.0049	0.7494 ± 0.0059	0.8362
Cross-bucket pooled	0.7854 ± 0.0013	0.7785 ± 0.0013	0.8694

Table 6.3: Per-domain HGT baseline 5-fold CV results. Motor accidents is the easiest domain; sexual offences the hardest.

The ranking of domains by difficulty is stable across every ablation configuration: *Motor Accidents* > *Land & Property* $\approx$ *Family & Matrimonial* > *Financial Fraud* > *Sexual Offences*. This ordering tracks two structural properties identified in Chapter 5: (i) motor-accidents cases concentrate on a very small set of provisions and courts, producing highly repetitive, high-connectivity sub-graphs that the GNN generalises over easily; (ii) sexual-offences and financial-fraud carry the largest outcome ambiguity at the summary level (hearings frequently remand or partially dispose, which complicates the binary-outcome target produced by the Mistral labeller) and the smallest per-case sentence-level structural signal.

The cross-bucket pooled dataset is *harder* than the average of the per-domain datasets (0.7854 vs. 0.802 mean): pooling exposes the model to domain-specific label noise and cross-domain distribution shift that each single-domain model can avoid. The `party_args_lr_decay` variant closes most of this gap.

One additional point is worth making explicit: the best global ablation is not the universal best single-domain ablation. Table 6.4 shows that the pooled optimum is a compromise configuration, while the best domain-specific choice depends on the local structure of each bucket.

Dataset	Best acc. variant	Acc.	Best macro-F1 variant	Macro-F1
Cross-bucket pooled	<code>party_args_lr_decay</code>	0.803	<code>party_args_lr_decay</code>	0.795
Family & Matrimonial	<code>no_names</code>	0.823	<code>hierarchical_enc</code>	0.804
Financial Fraud	<code>baseline</code>	0.777	<code>no_names</code>	0.765
Land & Property	<code>party_args_lr_decay</code>	0.820	<code>party_args_lr_decay</code>	0.802
Motor Accidents	<code>case_node_minimised</code>	0.834	<code>party_args_lr_decay</code>	0.797
Sexual Offences	<code>party_args_lr_decay</code>	0.779	<code>hierarchical_enc</code>	0.756

Table 6.4: Best-performing variant by dataset under accuracy and macro-F1. The pooled winner is not uniformly dominant once the graph is restricted to a single legal domain.

6.8 Ablation Findings

The main ablation numbers are collected in Table 6.2 for the cross-bucket dataset and in Table 6.5 for the per-domain breakdowns.

Ablation	Family & Mat.	Fin. Fraud	Land & Prop.	Motor	Sex. Off.
<code>baseline</code>	0.816	0.777	0.816	0.830	0.771
<code>case_node_min.</code>	0.811	0.766	0.815	0.834	0.769
<code>no_names</code>	0.823	0.775	0.815	0.831	0.774
<code>no_cross_case</code>	0.818	0.775	0.817	0.829	0.776
<code>text_only</code>	0.814	0.769	0.816	0.821	0.772
<code>hierarchical_enc</code>	0.821	0.772	0.813	0.829	0.778
<code>section_sep_enc</code>	0.821	0.772	0.817	0.830	0.770
<code>party_args_lr_decay</code>	0.819	0.770	0.820	0.831	0.779

Table 6.5: Per-domain accuracy across ablations (5-fold mean).

The findings:

1. **Text dominates.** `text_only` is within 0.01 accuracy of the baseline on every bucket – the bulk of the signal is carried by the `bge-m3` embeddings on text nodes, with entity nodes contributing a smaller, bucket-dependent boost.
2. **Names are removable.** `no_names` matches or slightly beats the baseline. This is the operational leakage check: removing party and judge identities neither helps nor hurts, which is the signature of a model that is not using them as a shortcut.
3. **Cross-case sharing survives *removal* but underlies the hub structure.** `no_cross_case` matches baseline on the pooled dataset but

degrades slightly on financial-fraud and sexual-offences, the two buckets where statute and precedent sharing carry the most information per case.

4. **Section separation and hierarchical chunking help modestly and unevenly.** `section_sep_enc` and `hierarchical_enc` improve most on domains where the ARGUMENTS block is long (family-matrimonial, sexual-offences); they do not help on motor-accidents, where texts are short.
5. **party_args_lr_decay is the best pooled configuration** on the cross-bucket dataset (+1.7 acc, +1.7 macro-F1 over baseline), suggesting that stabilising the party-and-argument sub-module prevents it from over-fitting to per-bucket surface cues while still benefiting from its signal.

6.9 Probing Classifier Analysis

To separate what the GNN *adds* from what is already present in the input features, a probing classifier (L2-regularised logistic regression, $C = 1$) is trained on three representations of the cross-bucket test set:

Probe input	Dim	Accuracy	Macro-F1
Majority baseline	0	0.609	0.378
Pre-GNN raw features (case-level)	1036	0.744	0.736
Post-GNN case embedding (baseline)	64	0.800	0.795
Post-GNN case embedding (<code>no_names</code>)	64	0.798	0.794

Table 6.6: Probing classifier results on fold 0 of the cross-bucket dataset. The post-GNN embedding reaches probing accuracy within a few points of the full end-to-end model, and the `no_names` probe matches the baseline probe.

The +5.6-point gain from pre- to post-GNN embedding at equal probing capacity is the cleanest single measurement of the GNN’s contribution. The `no_names` probe being indistinguishable from the baseline probe confirms that the information the GNN adds is not party-identity information.

6.10 Attention and Saliency Analysis

Per-relation attention weights (`prel_values.json`) and per-node-type gradient saliencies are computed on a held-out fold for interpretability. Two consistent patterns emerge across all configurations.

The ARGUMENTS node is the hinge. Gradient saliency on the case-node logit (summed across the dataset) ranks the input node types as follows: CASE $\gg$ ARGUMENTS > OTHER_LAWYER_ARGUMENTS > RESPONDENT_ARGUMENTS > FACTS > JUDGE > PETITIONER_ARGUMENTS > COURT > PREAMBLE > PETITIONER $\approx$ PRECEDENT > STATUTE $\approx$ LAWYER $\gg$ PROVISION. The text nodes carrying the legal argument structure (the various ARGUMENTS variants plus FACTS) dominate, followed by the bench and court identity; individual statutes and provisions contribute less per-node but do so at scale.

Reverse edges dominate attention mass. Per-relation attention scores (first- and second-layer HGT) are clustered tightly around 1, with the `rev_has_*` edges – those that carry information *from* text and entity nodes *back to* the case node – systematically higher than their forward counterparts. This is consistent with the architecture: the classifier reads only the CASE node’s final representation, so the value of a neighbour is measured by how strongly the reverse edge delivers it.

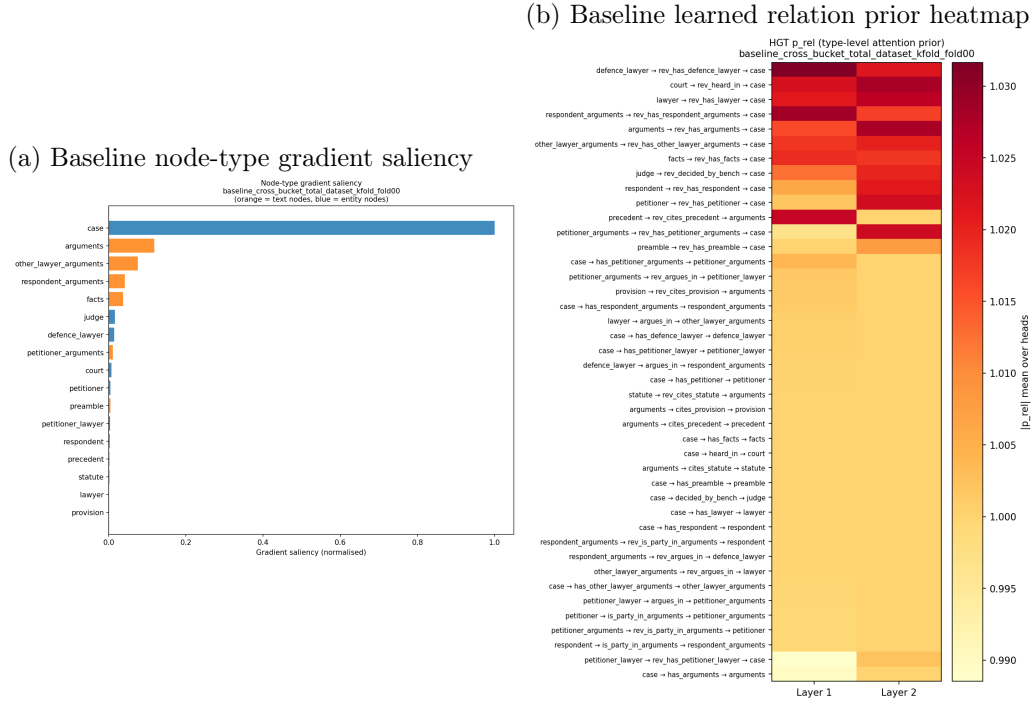


Figure 6.1: Interpretability plots for the baseline cross-bucket model on fold 0. The left panel visualises which node types contribute most to the case logit under gradient saliency; the right panel visualises the learned per-relation attention prior.

Figure 6.1 makes the interpretability claims visual. The model is overwhelmingly driven by the case representation and the argument-bearing text nodes, not by a single authority type. At the same time, the relation-prior heatmap shows that the strongest learned edge types are reverse edges flowing back into the CASE node from text and contextual nodes. That combination helps explain two earlier findings at once: why **text_only** remains surprisingly strong, and why stripping names leaves performance almost unchanged. The model’s main computation is structured aggregation of textual legal context into the case node, not retrieval of identity cues.

6.11 SHAP Feature Importance

Because the post-GNN case embedding is only 64-dimensional and the downstream classifier is a logistic regression, a SHAP-style attribution reduces to the absolute coefficient of each latent dimension. The top dimensions on the baseline model (dim 16, 38, 13, 25, 29, 14, 0, 10, 27, 4) each carry $|\beta| \geq 0.14$; the remaining 54 dimensions contribute smaller, more distributed weight. No single dimension dominates, which is evidence that the case embedding is distributed rather than collapsing onto a small set of predictive features.

6.12 Embedding Visualisation (t-SNE)

t-SNE projections of the pre-GNN and post-GNN case embeddings, coloured by bucket (`tsne*_by_domain.png`) and by outcome label (`tsne*_by_label.png`), complete the picture. Three patterns appear:

1. Pre-GNN embeddings cluster strongly by *bucket*: the **bge-m3** representation of the concatenated text already separates the five legal domains. Label separability is weak at this stage.
2. Post-GNN embeddings retain the bucket clusters but, within each cluster, label-colour separation is visibly improved. The HGT’s contribution is therefore primarily *within-domain* label structure, not between-domain re-organisation.
3. The **no_names** model’s t-SNE is qualitatively indistinguishable from the baseline’s, consistent with the probing result and the ablation result: identity information is not what the model is using to form the embedding.

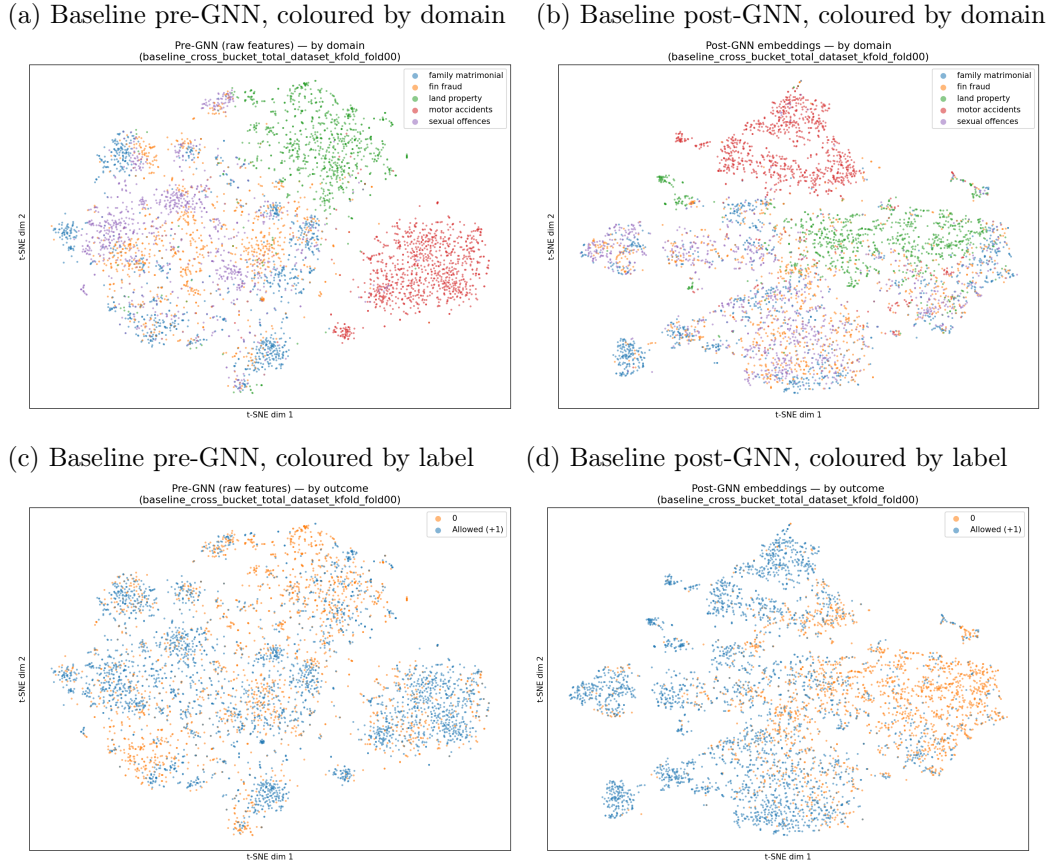
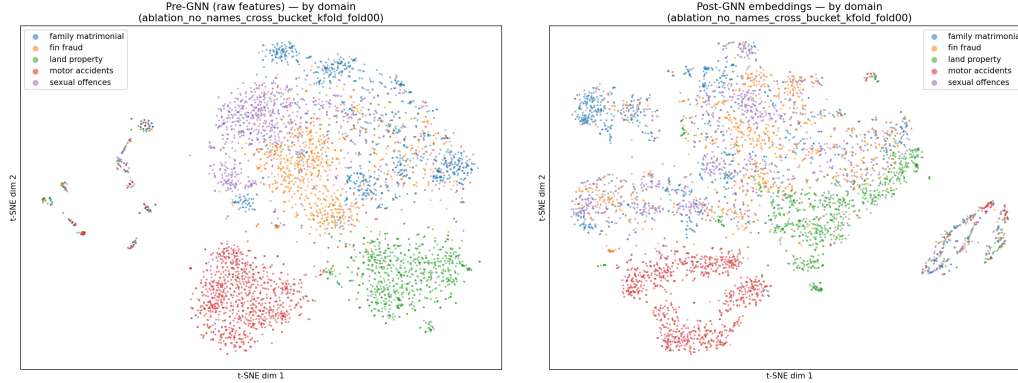


Figure 6.2: Baseline fold-0 t-SNE projections before and after graph message passing. The top row tracks domain structure; the bottom row tracks outcome structure.

Figure 6.2 makes the geometry of the embedding transition easier to read than the summary bullets alone. Before graph propagation, the representation is already domain-aware: motor-accidents and land-property form visibly coherent clusters, while family, fraud, and sexual-offences overlap more heavily. After graph propagation, these domain regions are preserved, but the label-coloured view becomes more structured inside them, especially in the larger motor-accident and land-property regions. This is visually consistent with the probing result: the GNN is not erasing domain information, but reorganising each domain locally in a more outcome-informative way.

(a) `no_names` pre-GNN, coloured by domain (b) `no_names` post-GNN, coloured by domain



(c) `no_names` pre-GNN, coloured by label (d) `no_names` post-GNN, coloured by label

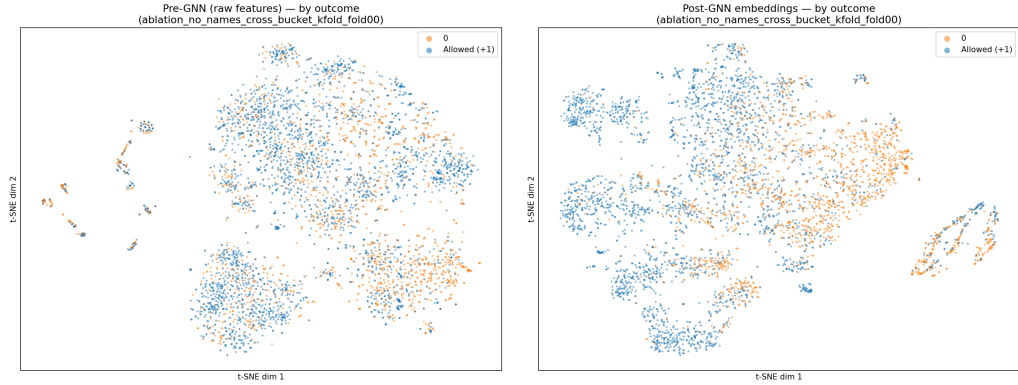


Figure 6.3: Corresponding fold-0 t-SNE projections for the `no_names` ablation.

The `no_names` t-SNEs in Figure 6.3 reinforce the ablation result from a different angle. Removing names changes the exact geometry of the projection, but it does not collapse the domain structure and it does not remove the post-GNN improvement in label organisation. In other words, the model still finds a structured outcome space after graph propagation even when party, lawyer, and judge names are withheld. The visual similarity is not exact – t-SNE should not be over-read at that level – but the qualitative conclusion is the same as in the accuracy tables: identity cues are not the dominant organising principle of the learned case embedding.

6.13 Error Analysis

Error analysis across ablations. The baseline’s per-domain error pattern – family-matrimonial 0.802, fin-fraud 0.754, land-property 0.778, motor-accidents 0.821, sexual-offences 0.777 – is nearly identical to the `no_names` model’s (0.797, 0.750, 0.782, 0.807, 0.770), reinforcing that the two variants share a prediction policy up to small, within-noise differences. The common pattern across every ablation – motor-accidents easiest, sexual-offences hardest – is a property of the data, not of the model.

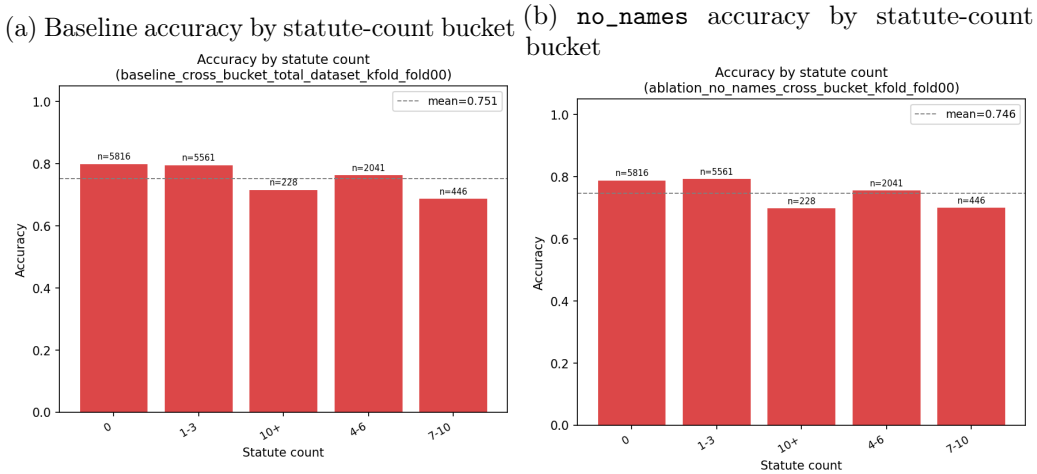


Figure 6.4: Held-out fold-0 accuracy as a function of the number of statute nodes attached to a case, for the baseline and `no_names` models.

Figure 6.4 adds an error pattern that is not visible in the domain-level accuracy tables. Accuracy is highest when a case cites few statutes and falls once the authority neighbourhood becomes dense: in the baseline, cases with 0 or 1–3 statute nodes sit near 0.80 accuracy, while the 7–10 and 10+ buckets are closer to 0.69–0.71. The `no_names` model shows essentially the same curve. This suggests that many of the residual errors are concentrated in doctrinally crowded cases rather than in identity-sensitive ones. That is an important distinction: the hard cases are not those where the model lacks names, but those where several statutory frames and remedial pathways coexist in the same judgment.

(a) Baseline accuracy by facts-length bucket (b) **no_names** accuracy by facts-length bucket

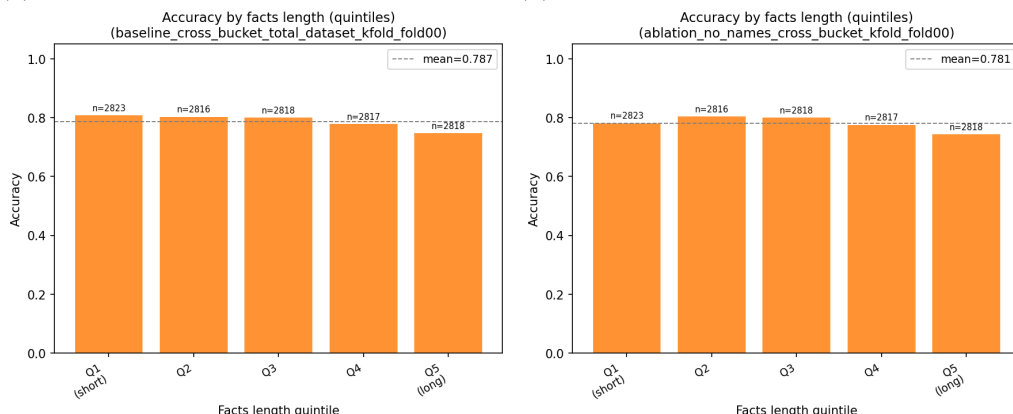


Figure 6.5: Held-out fold-0 accuracy as a function of the length of the facts section, grouped into quintiles from shortest to longest, for the baseline and **no_names** models.

Figure 6.5 is useful for a different reason than Figure 6.4: it tests whether the remaining errors are concentrated in structurally long judgments. They are. In the baseline, accuracy falls from about 0.81 on the shortest-facts quintile to about 0.75 on the longest-facts quintile; the **no_names** model shows nearly the same decline. This matters because it links the error analysis back to the ablation story. The modest, domain-specific gains from **hierarchical_enc** are easier to interpret once the failure pattern is visible: longer factual narratives do create a harder representation problem, but the difficulty comes from document complexity rather than from named entities or party identity.

Chapter 7

Discussion

7.1 Summary of Findings

The work pursued a single operational question: can a heterogeneous knowledge graph, built from Indian court judgments and filtered to be structurally leakage-safe, support an outcome-prediction model whose accuracy is attributable to legal reasoning rather than to identity or decision shortcuts? The chapters above produced four findings that, taken together, answer that question affirmatively but with important qualifications.

The pipeline produces a legally coherent corpus. Across all five legal-domain buckets, the top-PageRank entities recovered by OpenNyAI NER and the canonicalisation stage are exactly the statutes, provisions, and courts a lawyer would expect: IPC and CrPC in criminal matters, the Motor Vehicles Act 1988 and Section 166 in motor-accident matters, the Land Acquisition Act 1894 and the Constitution of India in land-property matters, and the POCSO Act in sexual-offences matters (Chapter 5). The corpus is *legally faithful*, not merely statistically clean.

Equally important, the reasoning-focused pruning introduced in Chapter 4 does not collapse the graph into something too thin to learn from. Removing GPE/ORG/DATE nodes and forbidding shortcut edges from local identity proxies to the case representation still leaves a large, connected, and legally meaningful structure. That is a substantive design result in its own right: the leakage controls are restrictive without being destructive.

The graph is hub-dominated and cross-domain. The 880k-node cross-bucket graph is organised around a small set of super-hubs – IPC, CrPC, the Supreme Court, the Apex Court – each of which bridges all five legal domains. The four IPC/CrPC-driven buckets share a significant fraction of entities with each other; land-property partially detaches via its own statutory backbone. The cross-domain transfer setting this thesis studies is, by construction, the right setting for this graph.

The HGT model learns cross-case structural signal. A probing classifier trained on the 64-dimensional post-GNN case embedding reaches 0.800 accuracy on the cross-bucket test set, compared to 0.744 for the pre-GNN raw features. The +5.6-point gain at fixed probing capacity isolates the HGT’s contribution from the underlying `bge-m3` text features. The best end-to-end configuration (`party_args_lr_decay`) reaches 0.803 accuracy, 0.795 macro-F1, and 0.885 ROC-AUC on the pooled five-domain test set.

The updated ablations also clarify *what kind* of gain this is. The graph does help, but not primarily by replacing text with a pure hub-traversal signal: `text_only` remains strong, `no_cross_case` is close to baseline on the pooled task, and the t-SNE and probing analyses show that message passing mainly reorganises the already-strong text representation into a more outcome-informative case embedding. In this regime, the graph’s contribution is best described as structured aggregation of legal context, with cross-case shared authorities providing a secondary, domain-dependent benefit rather than the entire predictive signal.

The model does not rely on identity shortcuts. The `no_names` ablation – which strips party, lawyer, and judge name content – matches or slightly improves over the baseline on every bucket. The t-SNE geometry of the `no_names` embedding is qualitatively indistinguishable from the baseline’s, and its probing accuracy is within 0.2 points of the baseline probe. The leakage-prevention strategy of Section 4.3 – field-level removal, rhetorical-role filtering, phrase-level masking, temporal gating on shared entities, and keeping identity nodes local – is both necessary (without RR filtering the model could read the decision verbatim) and sufficient (with all gates in place, removing identity features is a no-op).

The updated error analysis sharpens that conclusion. The remaining difficult cases are not those where names have been removed; they are the cases

with the densest statute neighbourhoods and the longest factual narratives. That same pattern appears in the baseline and `no_names` variants. The residual errors therefore look more like failures on doctrinally crowded or document-heavy judgments than failures caused by withholding identity information.

Taken together, the evidence for resisting identity-based leakage shortcuts is now multi-layered rather than resting on a single ablation. At the schema level, courts, judges, and lawyers are kept local and shortcut edges from identity proxies are forbidden; at the text level, rhetorical-role and phrase filtering remove direct decision leakage; at the model level, `no_names` matches baseline; and at the representation level, probing, t-SNE, saliency, and error analysis all point to the same conclusion: the learned signal is driven primarily by legal text structure and shared authority nodes, not by recoverable person- or bench-specific identity cues.

7.2 Limitations

Several limitations bound the interpretation of these results.

Transductive evaluation assumes entity overlap at test time. Every result in Chapter 6 is produced under the transductive protocol, i.e. the full graph (including test cases’ entity neighbourhoods) is visible during training. This matches the realistic deployment setting (a new judgment is appended to an existing graph built from the same canonical vocabulary), but it does not directly measure inductive generalisation to cases that cite entirely unseen statutes, precedents, or courts. An inductive split – the hardest version, in which a whole calendar year or a whole court is held out – is the natural follow-on experiment; the per-year error analysis already suggests older cases are harder, which is a prior on how that split would behave.

Outcome labels are LLM-generated. The win/loss labels are produced by prompting a Mistral model on the OpenNyAI-produced summary of each case. The labels are therefore noisy with respect to the ground-truth disposition, and the noise is not uniform across buckets: sexual-offences and financial-fraud matters often produce multi-part dispositions that do not map cleanly onto a binary win/loss axis. The fact that these are also the two worst-performing buckets is at least partially a label-noise effect rather than

a modelling failure. A multi-label or ordinal target is produced by the same pipeline and is an obvious next experiment.

RPC/RLC filtering is a lower bound on leakage, not a guarantee.

The leakage-safety story combines a deterministic filter (rhetorical-role labels, field names, leakage phrases) with a structural argument (shared-entity types are law, not identity). Neither is complete: an RR classifier can mis-label a decisive sentence, a leakage phrase list is never exhaustive, and a sufficiently expressive GNN could in principle reconstruct a decisive fragment from non-decisive neighbours. The `no_names` result is evidence against one specific leakage mode (party identity) but not against all leakage modes. A manual audit of high-confidence correct predictions – sampled across buckets and rhetorical-role density – is the next concrete step.

The ablation ordering is stable but the margins are small. The best configuration beats the baseline by +1.7 points of accuracy on the pooled dataset; most other variants are within ± 1 point. The 5-fold error bars are of the same order, which means that while the *qualitative* findings (text dominates, identity is removable, motor-accidents is easiest) are robust, the *quantitative* ordering of the close ablations should be reported as a cluster of comparable configurations rather than a strict ranking.

The pooled optimum is not the universal optimum. The new per-dataset best-variant table in Chapter 6 shows that the strongest global configuration is not the best choice in every domain: family-matrimonial, motor-accidents, and sexual-offences each favour different ablations under accuracy or macro-F1. This means the pooled cross-bucket score should be read as a useful compromise over heterogeneous graph geometries, not as evidence that one architectural setting is intrinsically best for all legal subdomains.

7.3 Future Work

Several directions follow naturally from the findings and limitations above, in rough order of expected payoff.

1. Inductive and temporal splits. The transductive protocol is the easier regime; a calendar-year hold-out (train on pre-2020, test on post-2020) and a court hold-out (train on all but one High Court, test on that one) would measure inductive generalisation directly. The connectivity-score machinery in Chapter 5 already provides the right diagnostic for these splits: a well-generalising model should remain competitive on the hold-out distribution’s low-connectivity tail.

2. Multi-label and ordinal outcome targets. The Mistral labelling pipeline produces a multi-label variant alongside the single-label `win/loss` target used in this thesis. Moving the classifier head to a multi-label sigmoid (or to an ordinal regression over the $[-1, +1]$ score) would (i) absorb the disposition ambiguity currently pushed into the binary target and (ii) enable disposition-specific error analysis (e.g. *remanded* cases separately from *dismissed* cases), which the present evaluation cannot distinguish.

3. Attention-guided interpretability. The per-relation attention and per-node-type saliency analyses in Chapter 6 produce rankings but not per-case explanations. A natural extension is to attach, to each correct prediction, the top- k neighbour nodes contributing the most to the case-node hidden state, and have a domain expert check whether those neighbours are legally defensible as reasons. This is where the thesis’s insistence on reasoning-focused edges and shared statute nodes pays off: the explanations are exactly the objects a lawyer would expect to see cited.

4. Scaling the bucket inventory. Five legal domains is a deliberate scope; the pipeline is bucket-agnostic by construction, and adding new domains (tax, labour, consumer) would expand the cross-domain spine and, through shared IPC/CrPC hubs, improve signal on the existing buckets as well. The main engineering cost is the per-bucket Mistral labelling pass; the rest of the pipeline scales without modification.

5. Stronger identity-leakage audits. The `no_names` ablation is a single, coarse leakage probe. Finer probes – a judge-hold-out split to measure judge-identity leakage directly; a court-hold-out split to measure court-identity leakage; and a text-similarity-based nearest-neighbour attack to detect cases whose neighbours in the graph also share rare token n-grams – would tighten

the leakage-safety claim from “no evidence of identity shortcuts” to “identity shortcuts are quantitatively bounded”.

6. Connectivity- and difficulty-aware modelling. Two empirical regularities now point to a targeted next step. First, the pooled best ablation is not the best ablation inside every bucket. Second, the hardest held-out cases are the ones with long factual narratives or dense statute neighbourhoods. A natural extension is therefore a model that allocates capacity conditionally – for example, using richer hierarchical text encoding for document-heavy cases, stronger graph propagation for high-connectivity cases, or a domain-adaptive head once the bucket geometry is known. That would turn the descriptive error analysis of Chapter 6 into an explicit modelling strategy.

Taken together, these findings and directions support a narrow and defensible thesis: legal outcome prediction over Indian court judgments *is* improvable by a heterogeneous, cross-case graph representation; the improvement is modest but measurable; it survives an aggressive identity-leakage audit; and it is explained by genuine cross-case structural signal rather than by the pathological shortcuts that have made prior work in this space hard to trust.

More strongly, the combination of schema design, leakage-filtered text construction, locality constraints on identity nodes, and post-hoc ablation evidence shows that this framework does not merely claim to avoid identity shortcuts; it operationally resists them while remaining predictive. That is the main methodological contribution of the thesis, not just a side condition on the experimental setup.

Chapter 8

Conclusion

This thesis studied Legal Judgment Prediction for Indian court cases as a structured prediction problem rather than a plain document-classification task. It argued that credible legal prediction requires more than strong accuracy numbers: it requires a representation that preserves legally meaningful structure while actively resisting leakage from decision-bearing text and identity-based shortcuts.

To address this, the thesis introduced an end-to-end pipeline that converts raw Indian judgments into leakage-audited, graph-ready case records through entity extraction, rhetorical-role segmentation, outcome labelling, multi-hearing consolidation, and reasoning-focused graph construction. On top of this representation, a Heterogeneous Graph Transformer was used to model both the internal structure of each case and the shared legal links across cases through statutes, provisions, and precedents.

Across more than 73,000 Indian court cases spanning five legal domains, the proposed framework achieved strong pooled performance and consistently outperformed text-only reference settings. More importantly, the results and ablations suggest that the gains are not coming merely from surface-level text patterns, but from cross-case legal structure encoded in a controlled and auditable way.

The main contribution of this thesis is therefore methodological. It shows that legal outcome prediction over Indian judgments becomes more credible when the task is built around leakage-safe data construction, heterogeneous graph design, and explicit constraints on what information may be shared across cases. While the improvements are measured rather than dramatic, they support a clear conclusion: graph-based legal prediction is most useful

when it is not only accurate, but also structurally faithful and experimentally honest.

Future work can build on this foundation by improving leakage audits further, extending the outcome space beyond binary decisions, and designing models that adapt more explicitly to domain-specific difficulty and long, complex factual narratives.

Bibliography

- [1] Nikolaos Aletras, Dimitrios Tsarapatsanis, Daniel Preşiu-Pietro, and Vasileios Lamps. Predicting judicial decisions of the European Court of Human Rights: a Natural Language Processing perspective. *PeerJ Computer Science*, 2:e93, 2016.
- [2] Paheli Bhattacharya, Kaustubh Hiware, Subham Rajgaria, Nilay Pochhi, Kripabandhu Ghosh, and Saptarshi Ghosh. A comparative study of summarization algorithms applied to legal case judgments. In *Advances in Information Retrieval – ECIR 2019*, volume 11438 of *Lecture Notes in Computer Science*, pages 413–428. Springer, 2019.
- [3] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. LEGAL-BERT: The muppets straight out of law school. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2898–2904. Association for Computational Linguistics, 2020.
- [4] Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Katz, and Nikolaos Aletras. LexGLUE: A benchmark dataset for legal language understanding in English. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4310–4330. Association for Computational Linguistics, 2022.
- [5] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference 2020*, pages 2704–2710. ACM, 2020.
- [6] Abhinav Joshi, Shounak Paul, Avinash Anand, Md. Shad Akhtar, and Ashutosh Modi. IL-TUR: Benchmark for Indian legal text understanding

- and reasoning. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 4035–4049. ELRA and ICCL, 2024.
- [7] Prathamesh Kalamkar, Aman Tiwari, Astha Agarwal, Saurabh Karn, Smita Gupta, Vivek Raghavan, and Ashutosh Modi. Corpus for automatic structuring of legal documents. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 4420–4429. European Language Resources Association, 2022.
 - [8] Daniel Martin Katz, Michael James Bommarito, and Josh Blackman. A general approach for predicting the behavior of the Supreme Court of the United States. *PLOS ONE*, 12(4):e0174698, 2017.
 - [9] Shubham Khatri, Pooja Rani, Aaryan Mattoo, Bhanu Pratap Singh Rawat, Kshitij Gulati, Shouvik Sachdeva, Gaurav Gambhir, and Vikram Goyal. Legal judgment prediction with Indian court cases using graph neural networks. In *Proceedings of the 19th International Conference on Artificial Intelligence and Law*, pages 191–200. ACM, 2023.
 - [10] Vijit Malik, Rishabh Sanjay, Shubham Kumar Nigam, Kripa Ghosh, Shouvik Sengupta, Arnab Bhattacharya, and Ashutosh Modi. ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4046–4062. Association for Computational Linguistics, 2021.
 - [11] Dimitris Mamakas, Petros Tsotsi, Ion Androutsopoulos, and Ilias Chalkidis. Processing long legal documents with pre-trained transformers: Modding LegalBERT and Longformer. In *Proceedings of the Natural Legal Language Processing Workshop 2022*, pages 130–142. Association for Computational Linguistics, 2022.
 - [12] Masha Medvedeva, Michel Vols, and Martijn Wieling. Using machine learning to predict decisions of the European Court of Human Rights. *Artificial Intelligence and Law*, 28(2):237–266, 2020.

- [13] Masha Medvedeva, Martijn Wieling, and Michel Vols. Rethinking the field of automatic prediction of court decisions. *Artificial Intelligence and Law*, 31(1):195–212, 2023.
- [14] Shubham Nigam, Parth Mehta, Arnab Bhattacharya, and Prasenjit Majumder. NyayaAnumana & IndianLegalBench: Towards comprehensive Indian legal judgment prediction and reasoning. *arXiv preprint arXiv:2501.03745*, 2025.
- [15] Shubham Kumar Nigam, Noel Shallum, Kripabandhu Ghosh, and Arnab Bhattacharya. LegalSeg: Unlocking the structure of Indian legal judgments through rhetorical role labeling. *arXiv preprint arXiv:2501.09690*, 2025.
- [16] OpenNyAI. About OpenNyAI. <https://opennyai.org/about>, 2024. Accessed 2024.
- [17] OpenNyAI. OpenNyAI documentation. <https://opennyai.org>, 2024. Accessed 2024.
- [18] T.Y.S.S. Santosh, Vatsal Venkatkrishna, Saksham Argwal, and Matthias Grabmair. Leveraging precedents for legal judgment prediction in low-resource settings: A case study on the European Court of Human Rights. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 3887–3897. ELRA and ICCL, 2024.
- [19] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web – ESWC 2018*, volume 10843 of *Lecture Notes in Computer Science*, pages 593–607. Springer, 2018.
- [20] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Peng Cui, Philip S. Yu, and Yanfang Ye. Heterogeneous graph attention network. In *The World Wide Web Conference*, pages 2022–2032. ACM, 2019.
- [21] Haotian Wu, Lingwei Wei, Mingze Li, Wei Zhou, Yan Zhao, Luqing Hou, Dapeng Liu, and Jine Tang. Precedent-enhanced legal judgment prediction with LLM and domain-model collaboration. In *Proceedings of the*

2023 Conference on Empirical Methods in Natural Language Processing, pages 12060–12075. Association for Computational Linguistics, 2023.

- [22] Haoxi Zhong, Zhipeng Guo, Cunchao Tu, Chaojun Xiao, Zhiyuan Liu, and Maosong Sun. Legal judgment prediction via topological learning. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3540–3549. Association for Computational Linguistics, 2018.